Round-Optimal Byzantine Agreement without Trusted Setup*

Diana Ghinea[1][0000−0002−5294−9459], Ivana Klasovitá[2][0009−0007−1276−3979], and Chen-Da Liu-Zhang[3][0000−0002−0349−3838]

[1] diana.ghinea@hslu.ch, Lucerne University of Applied Sciences and Arts
[2] ivana.klasovita@hslu.ch, Lucerne University of Applied Sciences and Arts
[3] chen-da.liuzhang@hslu.ch, Lucerne University of Applied Sciences and Arts

Abstract. Byzantine Agreement is a fundamental primitive in cryptography and distributed computing, and minimizing its round complexity is of paramount importance. The seminal works of Karlin and Yao [Manuscript'84] and Chor, Merritt and Shmoys [JACM'89] showed that any randomized r-round protocol must fail with probability at least $(c \cdot r)^{-r}$, for some constant c, when the number of corruptions is linear in the number of parties, $t = \theta(n)$. The work of Ghinea, Goyal and Liu-Zhang [Eurocrypt'22] introduced the first *round-optimal BA* protocol matching this lower bound. However, the protocol requires a trusted setup for unique threshold signatures and random oracles.
In this work, we present the first round-optimal BA protocols without trusted setup: a protocol for $t < n/3$ with statistical security, and a protocol for $t < (1-\epsilon)n/2$ with any constant $\epsilon > 0$, assuming a bulletin-board PKI for signatures.

1 Introduction

The problem of Byzantine Agreement (BA) [10, 20, 23, 34, 35, 40] constitutes one of the fundamental building blocks in cryptography and distributed protocols. It allows a set of n parties to reach agreement on a common value even if t of the parties deviate from the protocol.

One of the key measures of efficiency in distributed protocols is their round complexity, which is the number of synchronous communication rounds that a protocol takes until it terminates. On the one hand, it is well known that $t + 1$ rounds are necessary [19, 24] and sufficient [19, 30, 39] when considering deterministic protocols. On the other hand, the foundational works of Ben-Or [4] and Rabin [41] showed that this bound can be circumvented with the use of randomization.

In randomized BA protocols, there is an inherent trade-off between the number of rounds incurred by the protocol versus their probability of failure. More precisely, it is known that any r-round randomized protocol must fail with probability at least $(c \cdot r)^{-r}$, for some constant c, when the number of corruptions is linear in the number of parties $t = \theta(n)$ [11, 14, 33].

* This is the full version of a paper that appeared at Eurocrypt 2026.

Despite the extensive line of works [1,13,23,25,26,34,38] focusing on improving the round complexity of BA, the optimal trade-off was only achieved recently by the work of Ghinea, Goyal and Liu-Zhang [32], which introduced the first *round-optimal* BA protocol that runs in $O(r)$ rounds and achieves agreement except with probability $(c \cdot r)^{-r}$, for some constant c. The protocol is resilient up to $t = (1-\epsilon)n/2$ corruptions, for any constant $\epsilon > 0$.

The protocol leverages the Expand-and-Extract paradigm [26] introduced by Fitzi, Liu-Zhang and Loss, which combines a complex deterministic protocol (called Proxcensus), followed by one single multi-valued common coin to achieve BA. This is in contrast to traditional approaches [23] that operate via a sequence of iterations, and execute one coin per iteration. Although the Expand-and-Extract paradigm makes use of a single common coin, it crucially relies on the fact that the common coin protocol is ideal: it distributes the same uniform random value (with no bias nor error) to all parties with overwhelming probability to obtain a round-optimal BA. To achieve this, the authors leverage a trusted setup for unique threshold signatures to implement a 1-round ideal common coin using a random oracle [9,36].

This motivates the question whether it is possible to achieve a round-optimal Byzantine Agreement without trusted setup nor random oracles, or whether these assumptions are inherent.

Is there an r-round BA protocol without trusted setup nor random oracles that achieves agreement except with probability $(c \cdot r)^{-r}$, for some constant c, and secure up to some $t = \theta(n)$ corruptions?

Before stating our results, we note that all current r-round BA protocols without trusted setup or random oracles achieve a success probability of at most $1 - 2^{-r}$. This holds even for any fraction $t = \theta(n)$ of static corruptions.

1.1 Our Contributions

We answer this question in the affirmative by introducing the first round-optimal BA protocols without trusted setup nor random oracles. Our protocols achieve simultaneous termination (all parties simultaneously terminate in the same round) and are secure against a strongly rushing adaptive adversary.

Our first protocol achieves unconditional security and is secure up to $t < n/3$ corruptions.

Theorem 1. *There is an $O(r)$-round BA protocol tolerating an unbounded adversary corrupting up to $t < n/3$ parties, achieving agreement except with probability $(c \cdot r)^{-r}$, for some constant c.*

Our second protocol assumes a bulletin-board PKI[4] and is secure up to $t < (1-\epsilon)n/2$ corruptions, for any constant $\epsilon > 0$.

[4] In a bulletin-board PKI the keys from corrupted parties can be chosen adversarially. See [8] for a nice discussion.

Theorem 2. *Assuming a bulletin-board PKI, there is an $O(r)$-round BA protocol tolerating a polynomially-bounded adversary corrupting up to $t < (1-\epsilon)n/2$ corruptions, for any constant $\epsilon > 0$, achieving agreement except with probability $(c \cdot r)^{-r}$, for some constant c.*

1.2 Related Work

We give an overview of the progress in the round complexity of BA protocols, focusing on the binary-input case. To achieve multi-valued input BA, one can use standard extension protocols [43], at the cost of an additional 2 rounds in the $t < n/3$ case, and 3 rounds in the $t < n/2$ case.

The foundational work of Feldman and Micali [23] introduced an unconditional protocol for $t < n/3$ with expected constant number of rounds. This was later extended to the $t < n/2$ setting by Fitzi and Garay [25] using number-theoretic assumptions, and Katz and Koo [34] assuming a bulletin-board PKI. Abraham et al. [1] further extended the above results to also achieve expected $O(n^2)$ communication complexity. All these protocols achieve agreement in $O(r)$ rounds except with probability 2^{-r}.

Assuming an ideal 1-round coin-flip protocol with no error nor bias (which can be achieved using unique threshold signatures and random oracle [9,36]), the protocol of Feldman and Micali [23] for $t < n/3$ and Micali and Vaikuntanathan [38] for $t < n/2$ achieve agreement in $2r$ rounds except with probability 2^{-r}.

In the same setting with an ideal coin-flip protocol, Fitzi, Liu-Zhang and Loss [26] introduced the Expand-and-Extract paradigm (which can be seen as a generalized version of the Feldman and Micali iteration paradigm), and introduced a protocol with $r+1$ rounds for $t < n/3$, and $\frac{3}{2}r$ for $t < n/2$, to achieve agreement except with probability 2^{-r}. The result was later improved by Ghinea, Goyal and Liu-Zhang [32] who introduced an $O(r)$-round protocol achieving agreement except with probability $(c \cdot r)^{-r}$ for constant c, and tolerating $t < (1-\epsilon)n/2$ corruptions, for any constant $\epsilon > 0$.

A line of work focused on achieving round-efficient solutions for *broadcast*, the single-sender version of BA, in the dishonest majority setting [2,12,27,29,42,44,45].

Karlin and Yao [33], and also Chor, Merritt and Shmoys [14] showed that any r-round randomized protocol must fail with probability at least $(c \cdot r)^{-r}$, for some constant c, when the number of corruptions, $t = \theta(n)$, is linear in the number of parties. This bound was extended to the asynchronous model by Attiya and Censor-Hillel [3].

Protocols with *expected* constant round complexity have probabilistic termination, where parties (possibly) terminate at different rounds, which introduce challenges when composing them. Several works investigated parallel composition [4,25], sequential composition [37], and universal composition [15,16]. Cohen et al. [17] showed lower bounds for Byzantine Agreement with probabilistic termination. The authors give bounds on the probability to terminate after one and two rounds: for a large class of protocols and a combinatorial conjecture, the halting probability after the second round is $o(1)$ (resp. $1/2 + o(1)$) for the case where there are up to $t < n/3$ (resp. $t < n/4$) corruptions.

2 Technical Overview

We summarize the techniques we used in our protocols.

2.1 Modified Expand-and-Extract

Our starting point is the protocol by Ghinea, Goyal and Liu-Zhang [32]. The protocol follows the Expand-and-Extract paradigm introduced by Fitzi, Liu-Zhang and Loss [26], which consists of three steps executed in sequence: expansion, multi-valued coin-flip, and extraction.

In the expansion step, parties jointly execute a deterministic protocol called ℓ-slot Proxcensus, for $\ell \geq 2$. In this protocol, every P_i inputs their input bit $b_i \in \{0,1\}$, and obtains as output a value $z_i = \texttt{Prox}_\ell(x_i) \in \{0,\ldots,\ell-1\}$. Proxcensus guarantees that if every honest party has the input $x_i = 0$ (resp. $x_i = 1$), then every honest party outputs $z_i = 0$ (resp. $z_i = \ell - 1$). Moreover, the honest parties outputs always lie within two consecutive indices, i.e., there is a value $v \in \{0,\ldots,\ell-2\}$ such that each honest P_i outputs $z_i \in \{v, v+1\}$.

The multi-valued coin-flip protocol gives a uniform common random value $c \in \{0,\ldots,\ell-2\}$ to all parties.

In the extraction step, the idea is to perform a cut, where the output bit is $y_i = 0$ if $z_i \leq c$, and $y_i = 1$ otherwise. See Fig. 1.

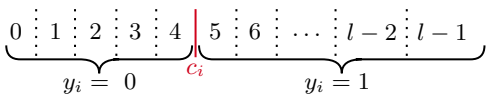

Fig. 1. Party P_i outputs a slot-value in $\{0,\ldots,\ell-1\}$, and the coin can "cut" the array of slots in any of the $\ell-2$ intermediate positions (indicated with dotted lines). If the obtained value lies on the left of the cut made by c_i (indicated with a red line), the output is $y_i = 0$. Otherwise, the output is $y_i = 1$.

The rationale is that since honest parties lie within two consecutive slots, there is only one value of c (out of $\ell-1$ values) that may split the honest parties into outputting different bits. Moreover, if the honest parties start with the same bit, they are guaranteed to output that bit.

This means that the error probability of this protocol is $1/(\ell-1)$, assuming the coin is perfect. The protocol in [32] introduces a $(3r)$-round Proxcensus with $\ell = (c \cdot r)^r$ for some constant c, which along with a 1-round ideal coin-flip leads to the desired result. The protocol is resilient up to $t = (1-\epsilon)n/2$, for any $\epsilon > 0$, and uses a bulletin-board PKI (to instantiate the Proxcensus), as well as unique threshold signatures and random oracles (to instantiate the coin).

We note that one can also achieve a Proxcensus protocol secure against $t < n/3$ in the plain model that provides similar guarantees by replacing one of the building blocks in the protocol of [32] (see Appendix A).

Our Modification. The first task is to get rid of the 1-round ideal coin-flip based on unique threshold signatures and random oracles. Note that using instead an information-theoretic weak common coin [23, 34] that outputs the same random value only with constant probability p would not work, since this would introduce an error of $1 - p$ into the protocol above.

Instead, our first observation is to relax the coin so that with very high probability it provides the honest parties an output from a *uniform interval* of indices. We call this protocol $(\ell - 1, \delta)$-`ProxCoin`. More precisely, the protocol guarantees that all honest parties output a value $c_i \in [0, \ell - 2]$ (as we need), but the outputs from honest parties may differ in δ units – that is, $|c_i - c_j| \leq \delta$. Moreover, at least one honest party outputs a coin that is uniformly distributed over $[0, \ell - 2]$. It is easy to see that the error of the BA protocol becomes $\frac{1+2\delta}{\ell-1}$.

Even though the error is higher, we manage to achieve the result by introducing $O(r)$-round $(\ell-1, \delta)$-`ProxCoin` protocols with $\delta = t$ and $\ell = (c \cdot r)^r$ for a constant c. See Section 4 for a more complete description of this approach.

2.2 Construction of `ProxCoin`

The construction of `ProxCoin` lets each party P_i choose a uniform random value $x_i \in [0, \ell - 2]$ and distributes it via a *conditional* verifiable secret sharing protocol `cVSS`. In the sharing phase of `cVSS`, each party P_i outputs a flag $\texttt{happy}_i$ and a share s_i. Then, if any honest party P_i sets $\texttt{happy}_i = \texttt{true}$, the protocol achieves all the guarantees from a standard verifiable secret sharing scheme (for an honest dealer, it is guaranteed that every honest P_j sets $\texttt{happy}_i = \texttt{true}$). This notion of `cVSS` is similar to the one considered by Feldman and Micali [23], but with a weaker consistency condition.

The idea is then to reconstruct all the values that were shared via `cVSS` and take the sum. This approach by itself does not work because a corrupted dealer $\widetilde{D}$ can distribute shares so that each honest party P_i ends up with $\texttt{happy}_i = \texttt{false}$. In this case, $\tilde{D}$ can choose their reconstructed value based on the values reconstructed from honest dealers (since `cVSS` does not guarantee any binding property in this case), arbitrarily biasing the common coin.

Instead, following the idea of Freitas et al. [28], the parties compute a *weighted* sum of the reconstructed values, where the weight of each party is (approximately) agreed upon. That is, the final coin for party P_i is $c_i = \left\lceil \sum_j w_j \cdot x_j \right\rceil$, where w_j is the computed weight for P_j's value and x_j is the reconstructed value from the `cVSS` where P_j is the dealer. The idea here is that the protocol guarantees that (i) every honest party P_i sets $w_j = 1$ for every honest dealer P_j, (ii) every corrupted P_j must choose their values x_j independently of the honest values and thus cannot influence the common coin, (iii) the weights w_j computed for each corrupted P_j have a bounded small difference, and (iv) if an honest party obtains $w_j > 0$, then the parties successfully reconstruct the same value shared by P_j.

To achieve this, the weights of the parties are computed using a `Prox` protocol, where each party P_u inputs their flag bit $\texttt{happy}_i$, obtaining an index between $[0, \ell - 1]$ (honest indices differ by at most 1). This index is scaled to be a number

between $[0, 1]$, simply by dividing by $\ell - 1$. This achieves the required guarantees stated above. Note that if the weight is positive for any party P_j, there was one honest party that input flag `true` (meaning the value that was distributed achieves the VSS guarantees, so this value was chosen independently of the other values, and can be reconstructed). Moreover, due to the guarantees from `Prox` and `cVSS`, weights for honestly dealt values are 1 and values dealt by corrupted dealers have a weight that differs by at most $1/(\ell - 1)$. The honest parties' final coins will then differ by at most $\delta \leq \lceil t \cdot (\ell - 2)/(\ell - 1) \rceil \leq t$, since every value in the weighted sum is between $[0, \ell - 2]$.

2.3 Protocols for Conditional Verifiable Secret Sharing

All that is left is to describe how to achieve *conditional* verifiable secret sharing. We propose two separate protocols – one in the plain model, secure against $t < n/3$ corruptions, and one assuming PKI and digital signatures, secure against $t < n/2$ corruptions. In the following, we discuss these in detail.

Protocol `cVSS` for $t < n/3$ with Perfect Security. Our starting point is the protocol by Gennaro, Ishai, Kushilevitz, and Rabin [31] that achieves VSS with perfect security. The sharing phase of the protocol proceeds as follows:

- *Step 1.* First, the dealer D chooses a bivariate polynomial $f(x, y)$ of degree at most t in each variable, such that $f(0, 0) = s$ and sends to each party P_i the i-th projections $a_i(x) = f(x, i)$ and $b_i(y) = f(i, y)$. Moreover, each party P_i sends to each P_j a uniform random pad r_{ij}.
- *Step 2.* Each party P_i broadcasts the blinded values $a_{ij} = a_i(j) + r_{ij}$ and $b_{ij} = b_i(j) + r_{ji}$, where r_{ji} denotes the pad that P_i receives from P_j.
- *Step 3.* For each conflicting pair ($a_{ij} \neq b_{ji}$), the involved parties broadcast their local points: P_i broadcasts $a_i(j)$, P_j broadcasts $b_j(i)$ and D broadcasts $f(j, i)$. A party is then considered *flagged* if their value does not match D's value. If there are more than t flagged parties, D is disqualified.
- *Step 4.* For each flagged party, D broadcasts the polynomial $a_i(x)$, and each party P_j who is unflagged broadcasts $b_j(i)$.
- *Local Computation.* Every party checks whether Step 4 has provided sufficient evidence supporting the flagged parties or D in the conflicts observed in Step 3. If there is sufficient evidence against D, then D is disqualified.

In the reconstruction phase, every party P_i that has completed the sharing phase as unflagged broadcasts $s_i = a_i(0)$. Then, for every party marked as flagged, take $s_i = a_i(0)$ where $a_i(x)$ is the polynomial broadcasted by D in round 4. Let $f'(y)$ be the t-degree polynomial resulting from the Reed-Solomon error-correction interpolation procedure on the shares $s_1, \ldots, s_n$, and output $f'(0)$.

If D is honest, all honest parties remain unflagged, since their broadcasted points always match D's broadcasted points. As a consequence, there are at most t flagged parties and D is not disqualified. The shares of the honest parties then

allow for the correct reconstruction of the dealer's secret s. On the other hand, if D is corrupted and was not disqualified during the sharing phase, the shares from unflagged parties completely determine the value that can be reconstructed. This is because there are at least $n-t$ unflagged parties, out of which at least $n-2t \geq t+1$ are honest, so their polynomials $b_i(y)$ uniquely determine a degree-t bivariate polynomial $f'(x,y)$. Furthermore, it is not hard to see that the share s_i distributed during the reconstruction for each honest party P_i (including flagged honest parties), will satisfy $s_i = f'(0,i)$.

Our $\texttt{cVSS}$ construction follows the outline of [31], but replaces broadcast with a weaker primitive, called Graded Broadcast GBC. If a sender S distributes a message m via Graded Broadcast, the parties receive a message (if S is honest, this is m) with grade of confidence in $\{0,1,2\}$ (if S is honest, this is 2). Moreover, the honest parties' grades differ by at most 1, and agreement on the message received is guaranteed when all honest parties' grades are non-zero.

Graded Broadcast ensures that, when D is honest, the honest parties' view during the sharing phase remains consistent – up to conflicts claimed by corrupted parties, as Graded Broadcast does not always provide agreement under a corrupted sender. Such conflicts will be solved regardless in favor of D, as D is honest, and VSS is achieved. Moreover, note that replacing broadcast by Graded Broadcast does not affect the secrecy guarantees – the adversary has no information about the secret before reconstruction.

The honest parties set their flag $\texttt{happy}$ to $\texttt{false}$ when they observe D deviating from the protocol: this includes receiving messages from D with grades lower than 2, but also reasons for disqualification in the VSS protocol of [31]. We note that we are unable to require the parties to disqualify D upon observing sufficient evidence that D is corrupted. Due to the weaker properties of Graded Broadcast, it is possible that an honest party has not observed this evidence, and believes that D is honest. Hence, upon observing evidence that D is corrupted, an honest party will simply set its flag $\texttt{happy} := \texttt{false}$. We only need to achieve VSS if at least one honest party has maintained $\texttt{happy} = \texttt{true}$, i.e., it has not observed evidence of D being corrupted. In such cases, our construction ensures that all honest parties obtain valid shares in the sharing phase, which uniquely determine the secret they obtain in the reconstruction phase – even if some of the honest parties have observed D to be corrupted. Conversely, if an honest party P was not provided with valid shares, then all honest parties will observe this – in such cases, P will be flagged – we make sure that all honest parties indeed flag P, observe that the conflict ends in P's favor, and set $\texttt{happy} = \texttt{false}$.

Protocol $\texttt{cVSS}_{\texttt{auth}}$ for $t < n/2$ assuming a Bulletin-Board PKI. Our starting point is the VSS protocol of [18]. This protocol follows traditional verifiable secret sharing schemes [7, 22] with bivariate polynomials, but uses so-called *information-checking* (IC) signatures, instead of error correction. One can think about such signatures as information-theoretic signatures that can only be forwarded once. We note that our final Byzantine Agreement protocol in the honest-majority setting assumes a bulletin-board PKI for signatures, and

therefore our conditional VSS construction in this setting also relies on standard digital signatures as opposed to IC-signatures.

We briefly revisit the protocol of [18]. In order to share a secret s, the dealer D creates a random bivariate polynomial $f(x, y)$ of degree at most t, with $f(0, 0) = s$. The univariate polynomial projections $f(x, i)$ and $f(i, y)$ are sent to party P_i in a signed manner (by sending all the points $(a_{1i}, \ldots, a_{ni}) = (f(1, i), \ldots, f(n, i))$ and $(b_{i1}, \ldots, b_{in}) = (f(i, 1), \ldots, f(i, n))$, where each point is signed using signatures). After this, the parties can bilaterally compare the cross-point values between them, and expose inconsistent behavior by D by broadcasting the signatures. If an inconsistency is detected, D is disqualified. After the cross-checking process, the values held by honest parties are consistent. Since there are at least $n - t \geq t + 1$ honest parties, these values uniquely define a bivariate polynomial $f'(x, y)$ of degree at most t, which in turn defines a fixed secret s' (which is $s' = s$ if D is honest). Therefore, this already ensures that D is committed to a value after the sharing phase. However, the adversary may still corrupt the secret at reconstruction time – the corrupted parties may send arbitrary shares in the reconstruction phase. To prevent this, each share that a party distributes during the reconstruction phase needs to be signed both by D and by other parties in the sharing phase.

Our strategy for achieving Conditional VSS in this setting will be similar to that described for the plain model: we follow the outline of [18], replacing each broadcast invocation with Graded Broadcast. Whenever an honest party observes evidence that D is corrupted, it may set its flag `happy` to `false`, as opposed to disqualifying D – this is important, as the honest parties might not have an identical view over D's behavior. This way, VSS is achieved whenever D is honest. For the case where D is corrupted, we make sure that, unless D provides the honest parties with valid shares that will uniquely determine the secret they obtain in the reconstruction phase, then all honest parties observe this during the sharing phase and set `happy` := `false`. This way, if D is corrupted and an honest party holds `happy` = `true` at the end of the sharing phase, then the honest parties are able to reconstruct the same secret in the reconstruction phase, hence VSS is achieved in this case as well.

3 Preliminaries

We are working in a setting with n parties $\mathcal{P} = \{P_1, P_2, \ldots, P_n\}$. We additionally consider a field $\mathcal{K}$ of size at least $n + 1$.

3.1 Communication

We consider a fully-connected network where parties are connected via point-to-point secure channels. The network is synchronous, meaning that the protocol execution proceeds synchronously in rounds, and messages sent at the start of a round are guaranteed to be delivered by the end of the round.

3.2 Adversary

We consider an adversary that can adaptively corrupt up to t parties at any point of the protocol's execution. Corrupted parties can deviate arbitrarily from the protocol. Moreover, the adversary is strongly rushing: they can observe the messages sent by honest parties in a round before choosing their own messages for that round, and, when an honest party sends a message during some round, it can immediately corrupt that party and replace this round's messages with another of its choice (as long as they have not been delivered yet).

3.3 Building Blocks

Signatures. Some of our protocols require the usage of a signature scheme. A signature scheme is a triple of efficient algorithms $(\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Ver})$, where:

- A key generation algorithm that outputs a pair containing the secret and public key $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\kappa)$.
- A signing algorithm Sign that takes as input the secret key sk and a message m, and produces a signature $\sigma \leftarrow \mathsf{Sign}_{\mathsf{sk}}(m)$.
- A verification function, which given the public key pk, a message m and a signature σ, outputs a bit $b \leftarrow \mathsf{Ver}_{\mathsf{pk}}(m, \sigma)$ indicating whether the signature verifies correctly.

The signature scheme is correct, meaning that $\mathsf{Ver}_{\mathsf{pk}}(m, \mathsf{Sign}_{\mathsf{sk}}(m)) = 1$ for every message m and keys generated $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\kappa)$.

In this paper, for clarity and ease of exposition, we treat signatures as ideal objects in all of our protocols [21], meaning that a computationally-bounded adversary cannot forge a valid signature without knowing the secret key. When replacing the signatures with real-world instantiations, the results hold against computationally-bounded adversaries except with negligible probability.

In our *authenticated* protocols, parties have access to a bulletin-board public key infrastructure (PKI) for signatures, where all parties hold the same vector of public keys $(\mathsf{pk}_1, \mathsf{pk}_2, \ldots, \mathsf{pk}_n)$, and each honest party P_i holds the secret key sk_i associated with pk_i.

Linear Error Correcting Code. We use standard Reed-Solomon (RS) codes with parameters $(n, t+1)$. There are two algorithms:

- Encoding. Given inputs $m_1, \ldots, m_{t+1}$, the encoding function outputs n codewords (a.k.a. shares) $(s_1, \ldots, s_n)$ of length n, such that any $t+1$ codewords uniquely determine the input message and the other codewords.
- Decoding. Given up to c errors and d erasures in codewords $(s_1, \ldots, s_n)$, one can reconstruct the original message $(m_1, \ldots, m_{t+1})$ if and only if $n - t - 1 \geq 2c + d$.

3.4 Primitives

We define the primitives we consider in the paper.

Byzantine Agreement. We present the definition of Byzantine Agreement with binary inputs below. Note that the *Termination* property is required implicitly.

Definition 1 (Byzantine Agreement). *A protocol Π where initially each party P_i holds an input value $x_i \in \{0,1\}$ and terminates upon generating an output y_i is a Byzantine Agreement protocol, resilient against t corruptions, if the following properties are satisfied whenever up to t parties are corrupted:*
Validity: *If all honest parties have input x, every honest party outputs $y_i = x$.*
Agreement: *Any two honest parties P_i and P_j output the same value $y_i = y_j$.*

Binary Proxcensus. We consider a variant of Byzantine Agreement, called Proxcensus [32], which allows parties to output an index value within an array of ℓ values, so that honest parties' outputs lie within two consecutive indices. Moreover, if parties have the same input bit, they all output an extremal index.

Definition 2 (Binary Proxcensus). *Let $\ell \geq 2$. A protocol Π where initially each party P_i holds an input bit $x_i \in \{0,1\}$ and parties terminate upon generating an output $y_i \in \{0,\ldots,\ell-1\}$ is an ℓ-Proxcensus protocol, resilient against t corruptions, if the following properties hold whenever up to t parties are corrupted:*
Validity: *If all honest parties input $x_i = 0$ (resp. $x_i = 1$), then every honest party outputs $y_i = 0$ (resp. $y_i = \ell - 1$).*
Consistency: *The outputs of any two honest parties P_i and P_j lie within two consecutive slots. That is, there exists a value $v \in 0,\ldots,\ell-2$ such that each honest party P_i outputs $y_i \in \{v, v+1\}$.*

ProxCoin. A protocol ProxCoin allows parties to toss a common *random* index from an array with ℓ indices, such that all honest parties output δ-close indices.

Definition 3 (ProxCoin). *Let $\ell \geq 2$ and $\delta \leq \ell$ be natural numbers. A protocol Π where parties terminate upon generating an output $y_i \in \{0,\ldots,\ell-1\}$ is an (ℓ,δ)-ProxCoin protocol, resilient against t corruptions, if the following properties are satisfied whenever up to t parties are corrupted:*
Consistency: *if one honest party outputs value x and another honest party outputs y, then $|x-y| \leq \delta$.*
One Uniform Randomness: *the value output by at least one honest party must be uniformly distributed over the domain $[0,\ell-1]$.*

Conditional Verifiable Secret Sharing. We propose a relaxed definition of Verifiable Secret Sharing (VSS), which we denote by *conditional* VSS. In this primitive, each party P_i outputs a share value s_i and a happiness flag $\texttt{happy}_i$. Intuitively, the guarantees are similar to that of VSS (in terms of correctness and secrecy), except that the binding property only holds when there is an honest party P_i with $\texttt{happy} = \texttt{true}$.

Definition 4 (Conditional `VSS`). *A two-phase protocol Π with a designated party D (called the dealer) is a Conditional VSS protocol resilient against t corruptions, if the following properties hold when up to t parties are corrupted:*

Syntax: *D initially holds an input s. Each honest party P_i holds a boolean $\texttt{happy}_i$ and a share $s_i \in S \cup \{\perp\}$ at the end of the first phase (called the* sharing *phase), and outputs y_i at the end of the second phase (called the* reconstruction *phase).*

Sharing-Validity: *If D is honest, then every party P_i holds $s_i \neq \perp$ and $\texttt{happy}_i = \texttt{true}$ at the end of the sharing phase.*

Cheating-Prevention: *If any honest party P_i holds share $s_i = \perp$ at the end of the sharing phase, then every honest party P_j holds $\texttt{happy}_j = \texttt{false}$.*

Correctness: *If D is honest, every honest party P_i outputs $y_i = s$ after the reconstruction phase.*

Secrecy: *If D is honest, the joint view of the byzantine parties after the sharing phase is independent of s.*

Conditional Binding: *If at least one honest party P_i holds $\texttt{happy}_i = \texttt{true}$ at the end of the sharing phase, the joint view of the honest parties at the end of the sharing phase defines a value y' such that all honest parties output y' after the reconstruction phase.*

Graded Broadcast. Our Conditional VSS constructions rely on Graded Broadcast. This is a weakening of broadcast allowing a sender to distribute a value, such that every party P_i obtains a pair of value-grade (y_i, g_i), as described below. For concrete constructions, see [5, 23, 26, 38].

Definition 5 (Graded Broadcast). *A protocol Π where a designated party S (called the sender) holds an input x, and each party P_i terminates upon obtaining an output (y_i, g_i) with $g_i \in \{0, 1, 2\}$, is a Graded Broadcast protocol resilient against t corruptions if the following hold when up to t parties are corrupted:*

Validity: *If S is honest, then every honest party P_i outputs $(y_i, g_i) = (x, 2)$.*

Graded Consistency: *If P_i and P_j are honest, they output (y_i, g_i) and (y_j, g_j) such that $|g_i - g_j| \leq 1$. Moreover, if $g_i > 0$ and $g_j > 0$, then $y_i = y_j$.*

4 Expand-and-Extract Paradigm via ProxCoin

We describe the Expand-and-Extract paradigm with the replacement of the ideal coin by a ProxCoin described in Section 2. In the protocol below, Prox_ℓ is an ℓ-Proxcensus protocol, and $\texttt{ProxCoin}_{\ell-1,\delta}$ is an $(\ell - 1, \delta)$-ProxCoin protocol.

Protocol: $\Pi^{\ell}_{\texttt{EE}}$

Let $\ell \geq 2$ be a natural number. The protocol is described from the point of view of party P_i, with input bit $x_i \in \{0, 1\}$.

1: $z_i := \texttt{Prox}_\ell(x_i)$; $c_i := \texttt{ProxCoin}_{\ell-1,\delta}$.
2: If $z_i \leq c_i$, output 0. Otherwise, output 1.

The honest parties output the same bit whenever either all honest values z_i are on the left side of every coin c_i, i.e., $\max_{P_i} z_i \leq \min_i c_i$, or all honest values z_i are on the right side of every coin c_i, i.e., $\min_{P_i} z_i > \max_i c_i$. This leads us to the following result.

Theorem 3 ([26], Adjusted). *Assume $t < n$. Let $\texttt{Prox}_\ell$ be an ℓ-slot Prox-census protocol secure up to t corruptions, and $\texttt{ProxCoin}_{\ell-1,\delta}$ be an $(\ell-1,\delta)$-ProxCoin protocol, secure up to t corruptions. Then, $\Pi_{\text{EE}}^{\ell,\delta}$ achieves binary Byzantine Agreement against an adaptive, strongly rushing adversary with probability $1 - \frac{1+2\cdot\delta}{\ell-1}$.*

Proof. **Validity.** If each P_i inputs 0 to $\texttt{Prox}_\ell$, they obtain $z_i := 0$. Moreover, since $c_i \geq 0$, all honest parties output 0. Similarly, if each P_i inputs 1 to $\texttt{Prox}_\ell$, they obtain $z_i := \ell - 1$. As $c_i \leq \ell - 2$, every honest party outputs 1.

Agreement. If all honest parties satisfy the same condition, either $z_i \leq c_i$ or $z_i > c_i$, Agreement is achieved. As `ProxCoin` achieves one-process randomness, there is one honest party that outputs a random index $c \in [0, \ell-2]$. By consistency, all honest parties' coins $c_i \in [c-\delta, c+\delta]$. Then, the protocol achieves Agreement with probability at least $\Pr_c[\max_i z_i \leq c-\delta \text{ or } c+\delta < \min_i z_i]$, hence at least $1 - \Pr_c[c \in [\min_i z_i - \delta, \max_i z_i + \delta)] \geq 1 - \frac{1+2\cdot\delta}{\ell-1}$.

5 Conditional VSS

In this section, we describe our constructions for conditional VSS. As mentioned in Section 3.4, this primitive is a weakening of verifiable secret sharing, where the binding property is only achieved when one of the honest parties holds $\texttt{happy} = 1$.

5.1 Perfect Security with $t < n/3$

We first describe a construction in the plain model without setup. The protocol achieves perfect security against $t < n/3$ corruptions, and is based on the VSS protocol by Gennaro, Ishai, Kushilevitz, and Rabin [31].

Theorem 4. *Protocol $\texttt{cVSS} = (\texttt{cVSS.Sh}, \texttt{cVSS.Rec})$ is a conditional VSS protocol with perfect security up to $t < n/3$ corruptions. The round complexity of $\texttt{cVSS}$ is $R_{\texttt{Sh}} = 10$ for the sharing phase and $R_{\texttt{Rec}} = 1$ for the reconstruction phase.*

Protocol Description. We describe the complete protocol below, using a Graded Broadcast protocol GBC as a building block. Note that GBC can be achieved for $t < n/3$ without setup [5, 23] in 3 rounds and $O(n^2)$ bits of communication.

Protocol: `cVSS.Sh`

Distribution: Let s be the dealer D's input. Every party P_i sets $\texttt{happy}_i := \texttt{true}$.

1: D chooses a random bivariate polynomial $f(x, y)$ over $\mathcal{K}$ of degree at most t in each variable, such that $f(0, 0) = s$. For each party P_i, D defines $a_i(x) := f(x, i)$ and $b_i(y) := f(i, y)$ and sends the polynomials a_i, b_i to P_i.

2: Party P_i checks whether it has received a_i, b_i from D such that a_i, b_i are polynomials of degree at most t. If not, it sets a_i, b_i to a default polynomial and sets $\texttt{happy}_i := \texttt{false}$.
3: For every P_j, P_i samples r_{ij} uniformly at random from $\mathcal{K}$ and sends r_{ij} to P_j
4: Let r_{ji} be the value P_i receives from P_j (or some default value if no value was received).

Consistency checks:

5: For every P_j, P_i computes $a_{ij} = a_i(j) + r_{ij}$ and $b_{ij} = b_i(j) + r_{ji}$. Then, P_i sends a_{ij}, b_{ij} to all parties via GBC.
6: Party P_i checks for each P_j:
If it has not received value $b_{ji} = a_{ij}$ with grade 2, it sends $a_i(j)$ via GBC.
If it has not received value $a_{ji} = b_{ij}$ with grade 2, it sends $b_i(j)$ via GBC.
7: Unless D observes $a_{ij} = b_{ji}$ both with grade 2 in Step 5, it sends $f(j, i)$ to all parties via GBC.
8: If P_i has observed $a_{kj} \neq b_{jk}$ with grades at least 1 for some P_j, P_k in Step 6; and P_i has not received a response from D for this pair with grade 2 in Step 7, P_i sets $\texttt{happy}_i := \texttt{false}$.

Flagging parties:

9: Party P_i flags party P_j if any of these conditions holds:
- In Step 6, P_i has received $a_j(k)$ from P_j with grade 2, and in Step 7 P_i has received $f(k, j) \neq a_j(k)$ from D with grade at least 1.
- In Step 6, P_i has received $b_j(k)$ from P_j with grade 2, and in Step 7 P_i has received $f(j, k) \neq b_j(k)$ from D with grade at least 1.

10: If P_i has flagged at least $t + 1$ parties, it sets $\texttt{happy}_i := \texttt{false}$.
11: Unless D has observed P_j to be honest (all messages have arrived with grade 2) and consistent (in Step 6, any gradecasted values $a_j(i)$ and $b_j(i)$ satisfy $a_j(i) = f(i, j)$ and $b_j(i) = f(j, i)$), it sends the polynomials $a_j(x) = f(x, j)$ and $b_j(x) = f(j, x)$ to all parties via GBC.
12: If P_i has flagged party P_j and P_i has not received from D the polynomials $a_j(x) = f(x, j)$ and $b_j(x) = f(j, x)$ with grade 2, P_i sets $\texttt{happy}_i := \texttt{false}$.
13: Unless party P_i flagged itself, it checks for each party P_j whether:
- In Step 6, P_i has received $a_j(k)$ with grade at least 1 from P_j, and in Step 7, P_i has received $f(k, j) \neq a_j(k)$ from D with grade at least 1.
- In Step 6, P_i has received $b_j(k)$ with grade at least 1 from P_j, and in Step 7, P_i has received $f(j, k) \neq b_j(k)$ from D with grade at least 1.

If any of these conditions holds, P_i sends the values $a_i(j), b_i(j)$ via GBC.

Determine share:

14: For each party P_j that P_i flagged, P_i checks that it has received $a_k(j), b_k(j)$ with grade 2 from at least $2t + 1$ unflagged parties P_k, such that $a_j(k) = b_k(j)$ and $b_j(k) = a_k(j)$. If this is not the case, it sets $\texttt{happy}_i := \texttt{false}$.
15: Party P_i outputs its share $s_i := a_i(0)$ and the flag $\texttt{happy}_i$.

Protocol: cVSS.Rec

1: Every party P_i sends $s_i = a_i(0)$ to all parties
2: For every party P_j that P_i has flagged in Step 9 of cVSS.Sh, P_i sets $s_j := a_j(0)$, where a_j is the polynomial that P_i has received from D with grade at least 1 via GBC in Step 11 of cVSS.Sh.
3: Let $f_i(x)$ be the polynomial with degree up to t resulting from the Reed-Solomon error-correction interpolation procedure on $s_1, \ldots, s_n$. Party P_i can then compute the secret $y_i = f_i(0)$.

Analysis. We split the proof of Theorem 4 into a sequence of Lemmas. We first note that the Syntax property follows immediately from the protocol description, and the Cheating-Prevention property holds trivially as the parties never output $\perp$. Then, the Sharing-Validity property follows from Lemma 1. Correctness is discussed in Lemma 3, and Secrecy is proven in Lemma 2. Afterwards, we show that Conditional Binding is satisfied in Lemma 5. It remains to discuss the round complexity – cVSS incurs one round of communication plus three sequential steps involving n parallel invocations of GBC for the sharing phase, and n parallel invocations of GBC for the reconstruction phase. As GBC can be achieved within 3 rounds of communication [5, 23], this leads to 10 rounds for the sharing phase, and 3 rounds for the reconstruction phase.

Lemma 1. *Assume that the dealer D is honest and holds secret s, and consider the polynomial f that D chooses in Step 1 of cVSS.Sh. Then, every honest party P_i completes cVSS.Sh with $a_i(x) = f(x, i)$, $b_i(y) = f(i, y)$ and $\texttt{happy}_i = \texttt{true}$.*

Proof. Since D is honest, every honest party P_i receives $a_i(x) = f(x, i)$, $b_i(y) = f(i, y)$ in Step 1 – these are t-consistent, thus P_i does not set a_i, b_i to a default polynomial and keeps $\texttt{happy}_i = \texttt{true}$. It remains to analyze the steps of cVSS.Sh where P_i may set $\texttt{happy} := \texttt{false}$ and show that $\texttt{happy}_i = \texttt{true}$ is maintained.

In Step 8, P_i sets $\texttt{happy}_i := \texttt{false}$ only if it observes $a_{kj} \neq b_{jk}$ in Step 5 and has not received a response for this pair from D with grade 2 in Step 7. We note that, if D receives $a_{kj} = b_{jk}$ with grades 2 each in Step 5, P_i receives $a_{kj} = b_{jk}$ with grade at least 1 each. Otherwise, if D does not receive such a pair with grade 2, it responds by sending $f(j, i)$ to all parties via GBC, and P_i receives this response with grade 2. Hence, P_i maintains $\texttt{happy}_i = \texttt{true}$ in this step.

In Step 10, P_i sets $\texttt{happy}_i := \texttt{false}$ only if it has flagged at least $t + 1$ parties in Step 9. P_i flags P_j if P_i has received $a_j(k)$ or $b_j(k)$ in Step 6 and a conflicting response from D in Step 7. However, if both D and P_j are honest, the polynomials they distribute match and are received with grade 2, hence P_i does not flag an honest party P_j. Thus, P_i flags at most t parties, and maintains $\texttt{happy}_i = \texttt{true}$.

In Step 12, P_i sets $\texttt{happy}_i := \texttt{false}$ if it has flagged some party P_j in Step 9 and it has not received the polynomials corresponding to P_j from D with grade 2 in Step 11. We note that, if P_i flags P_j, then D has not observed P_j to be honest in Step 11: P_i has received a polynomial from P_j with grade 2 in Step 6

that did not match D's response in Step 7. GBC guarantees that D has received the same polynomial from P_j in Step 7 with grade at least 1, hence D has not observed P_j to be honest, and therefore D sends the polynomials $a_j(x) = f(x, j)$ and $b_j(x) = f(j, x)$ to all parties via GBC. Then, P_i receives this response with grade 2 in Step 12, and therefore maintains $\texttt{happy}_i = \texttt{true}$.

It remains to discuss Step 14. We have already shown that honest parties do not flag honest parties when D is honest in Step 9, hence no honest party flags itself. Then, for every P_j that P_i has flagged in Step 9, P_i checks whether $2t + 1$ unflagged parties P_k have confirmed in Step 13 the values $a_j(k)$ and $b_j(k)$ claimed by D in Step 11. Since no honest party marks itself, every honest party P_k sends $a_k(j) = f(j, k) = b_j(k)$ and $b_k(j) = f(k, j) = a_j(k)$ in Step 13, hence agreeing with D. Then, P_i receives at least $\geq 2t + 1$ values $a_k(j), b_k(j)$ that agree with D, and thus it maintains $\texttt{happy}_i = \texttt{true}$. Consequently, P_i completes cVSS.Sh with $\texttt{happy}_i = \texttt{true}$, which concludes our proof.

Lemma 2. *cVSS achieves Secrecy.*

Proof. We assume that D is honest with input secret s, and we show that the joint view of the byzantine parties before the start of cVSS.Rec is independent from s. Let f denote the polynomial chosen by D in Step 1.

We first establish that the byzantine parties' views after Step 1 is independent from s. The byzantine parties' view at the end of Step 1 consists of the rows $a_i(x) = f(x, i)$ and columns $b_i(y) = f(i, y)$ where P_i is byzantine. Hence, these are the polynomials corresponding to at most t different rows and at most t different columns. However, this is not sufficient information to determine s: for any secret s', there is a polynomial $f'(x, y)$ of degree at most t in each variable such that $f'(0, 0) = s'$, $f'(x, i) = a_i(x)$ and $f'(i, y) = b_i(y)$ for every byzantine party P_i. Therefore, the byzantine parties' joint view at the end of Step 1 is independent from s. It remains to show that the further steps of cVSS.Sh do not reveal any additional information.

In Step 5, values a_{ij}, b_{ij} are masked by random values r_{ij}, r_{ji} and thus independent of $a_i(k), b_i(k)$: these remain secret to the byzantine parties if P_i and P_j are honest.

In Step 6, an honest party P_i publishes $a_i(j)$ or $b_i(j)$ only if it has not received a matching value with grade 2 from P_j in Step 5. GBC ensures that, if P_j is honest, its value is received with grade 2 by all parties. Moreover, since D is honest, P_i and P_j hold $a_j(i) = b_i(j) = f(i, j)$ and $a_i(j) = b_j(i) = f(j, i)$. Hence, if P_i publishes the value $a_i(j)$ or $b_i(j)$, P_j must be byzantine, and the published values are already part of the byzantine parties' joint view at the end of Step 1.

In Step 7, D only publishes $f(j, i)$ if it has not observed a pair $a_{ij} = b_{ji}$ with grade 2 in Step 5. As described for Step 6, since D is honest, it has provided the parties with values $a_i(j) = b_j(i) = f(j, i)$. Also, assuming P_i, P_j are honest, they both add the same value r_{ij} to $a_i(j)$ resp. $b_j(i)$ and thus $a_{ij} = b_{ji}$. In addition, since GBC ensures that honest parties' messages are received with grade 2, D receives $a_{ij} = b_{ji}$ both with grade 2 for every pair of honest parties P_i, P_j in Step 5. Hence, if D publishes $f(i, j)$ in Step 7, at least one of the two parties

P_i and P_j is corrupted, and therefore the value published was already in the corrupted parties' joint view at the end of Step 1.

We now discuss Step 11: D publishes the polynomials $a_j(x), b_j(x)$ only if it has observed P_j deviating from the protocol: D has either received a message from P_j with a grade ≤ 1 via GBC, or P_j has sent values in Step 6 contradicting the ones it has received from D. Therefore, D only published the polynomials $a_j(x), b_j(x)$ of byzantine parties P_j, hence no new information.

For Step 13, we note that an honest P_i flags a party P_j only if P_j is byzantine (when D is honest): as described in the proof of Lemma 6, P_i flags P_j if P_j has sent a value $a_j(k)$ or $b_j(k)$ in Step 6, and P_i has received a conflicting response from D for P_j in Step 7. However, if both D and P_j are honest, the polynomials they distribute match and are received with grade 2, hence P_i does not flag P_j. Then, in Step 13, an honest party P_i only reveals values $a_i(j), b_i(j)$ where P_j is byzantine, and therefore does not reveal any new information.

Lemma 3. *cVSS achieves Correctness.*

Proof. We assume that D is honest with input s, and we show that every honest party P_i outputs $y_i = s$ at the end of `cVSS.Rec`.

If f denotes the polynomial chosen by D in Step 1 of `cVSS.Sh`, Lemma 1 ensures that every honest party P_i completes `cVSS.Sh` with share $s_i = a_i(0) = f(0, i)$. Then, in `cVSS.Rec`, every honest party P_i receives $s_j = f(0, j)$ for every honest party P_j. In addition, for every party P_j that P_i has flagged in `cVSS.Sh`, P_i sets $s_j := a_j(0)$, where $a_j(x)$ is the polynomial published by D in Step 11 of `cVSS.Sh`: since D is honest, $s_j = a_j(0) = f(0, j)$. P_i sets $f_i(x)$ as the polynomial of degree at most t resulting from the Reed-Solomon error correction interpolation procedure on $s_1, \ldots s_n$. As $n - t \geq 2t + 1$ of these shares agree with $f(0, j)$, there is a unique polynomial of degree at most t that the honest parties can obtain, and this is f. Hence, every honest party P_i outputs $y_i := f(0, 0) = s$.

The lemma below notes an additional property of `cVSS.Sh` that will be helpful when discussing Conditional Binding.

Lemma 4. *Assume that an honest party P_i flags an honest party P_j in Step 9 of cVSS.Sh. Then, if some honest party P_l completes cVSS.Sh with $\mathtt{happy}_l = \mathtt{true}$, every honest party flags P_j.*

Proof. Since P_i has flagged P_j in Step 9 of `cVSS.Sh`, P_i has received $a_j(k)$ or $b_j(k)$ from P_j with grade 2 in Step 6. GBC ensures that every honest party P_k has received the same value with grade 2 from P_j in Step 6.

Moreover, since P_l completes `cVSS.Sh` with $\mathtt{happy}_l = \mathtt{true}$, P_l has observed a response from D to P_j with grade 2 in Step 7. Hence, all honest parties receive this response with grade at least 1. Since P_i flags P_j, this response does not match the value distributed by P_j in Step 6: $a_j(k) \neq f(k, j)$ or $b_j(k) \neq f(j, k)$. Hence, all honest parties observe this and flag P_j.

Lemma 5. *cVSS achieves Conditional Binding.*

Proof. We assume that at least one honest party P_i holds $\texttt{happy}_i = \texttt{true}$ at the end of cVSS.Sh, and show that the joint view of the honest parties at the end of cVSS.Sh defines a value y' such that all honest parties output y' in cVSS.Rec.

We first note that, since P_i completes cVSS.Sh with happy = true, P_i has flagged at most t parties. Moreover, Lemma 4 ensures that the honest parties flag the same honest parties. This implies that there is a set $\mathcal{H}'$ of at least $n-2t \geq t+1$ honest parties P_j that have obtained in cVSS.Sh polynomials $a_j(x), b_j(x)$ of degree at most t such that $a_j(k) = b_k(j)$ for every $P_k \in \mathcal{H}'$. In the following, we consider a subset $\mathcal{H} \subseteq \mathcal{H}'$ of size $t+1$, and define $f'(x,y)$ as the unique polynomial of degree at most t in each variable such that $f'(j,k) = a_k(j) = b_k(k)$ for every pair of parties P_j, P_k in $\mathcal{H}$. Since these are $(t+1)\cdot(t+1)$ constraints, there is a unique such polynomial f'.

Next, we show that the shares $a_j(x), b_j(x)$ of every unflagged honest party $P_j \in \mathcal{H}' \setminus \mathcal{H}$ agree with f'. The polynomial $a_j(x)$ has degree at most t and satisfies $a_j(k) = b_k(j) = f'(k,j)$ for every party $P_k \in \mathcal{H}$. Hence, $a_j(x)$ and $f'(x,j)$ are both polynomials of degree at most t that agree in $\left|\mathcal{H}\right| = t+1$ points, and therefore they are identical. Analogously, $b_j(x)$ is identical with $f'(j,x)$.

Lastly, we consider the polynomial $a'_j(x)$ of a flagged honest party P_j. In Step 14 of cVSS.Sh, P_i has observed at least $2t+1$ unflagged parties (honest and corrupt), hence at least $t+1$ unflagged honest parties P_k that confirmed $a_j(k) = b_k(j)$ and $a_k(j) = b_j(k)$. Therefore, $a_j(k) = b_k(j) = f'(k,j)$ for at least $t+1$ of the honest parties in $\mathcal{H}'$. Since $a_j(x)$ and $f'(x,j)$ are polynomials of degree at most t, it follows that $a_j(x)$ and $f'(x,j)$ are identical. Analogously, $b_j(x)$ is identical with $f'(j,x)$.

Hence, we have obtained that the honest parties' shares uniquely determine a polynomial $f'(x,y)$ of degree at most t in each variable. It remains to discuss cVSS.Rec: in cVSS.Rec, every honest party P_j obtains $s_k = f'(0,k)$ for every honest party P_k. Hence, P_j obtains $n-t \geq 2t+1$ distinct points that lie on f', plus at most t points that do not. Then, the Reed-Solomon error-correction procedure ensures that the honest parties obtain the same polynomial $f_l(x) = f'(0,x)$ and therefore output $y_l := f_l(0) = f'(0,0)$.

5.2 Computational Protocol Assuming PKI for $t < n/2$

We now present a construction secure against $t < n/2$ corruptions that assumes a bulletin-board PKI for signatures, and is based on the VSS protocol of Cramer et al. [18]. Our construction is described by the theorem below.

Theorem 5. *Assuming a bulletin-board PKI for signatures, protocol* $\texttt{cVSS}_{\texttt{auth}} = (\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}, \texttt{cVSS}_{\texttt{auth}}.\texttt{Rec})$ *is a Conditional VSS protocol secure up to* $t < n/2$ *corruptions. The round complexity of* $\texttt{cVSS}_{\texttt{auth}}$ *is* $R_{\texttt{Sh}} = 14$ *for the sharing phase,* $R_{\texttt{Rec}} = 3$ *for the reconstruction phase.*

Protocol Description. We describe the protocol below. Similarly to cVSS, we make use of a Graded Broadcast protocol $\mathsf{GBC}_{\mathtt{auth}}$ as a building block. $\mathsf{GBC}_{\mathtt{auth}}$

can be achieved for $t < n/2$ corruptions assuming a bulletin-board PKI in 3 rounds and $O(n^3\kappa)$ bits of communication [26, 38].

Protocol: $\texttt{cVSS}_{\texttt{auth}}\texttt{.Sh}$

Distribution Let s be the dealer D's input. Every party P_i sets $\texttt{happy}_i := \texttt{true}$.

1: D chooses a random bivariate polynomial $f(x, y)$ of degree at most t in each variable, such that $f(0, 0) = s$. D sends to party P_i the values $a_{1i} = f(1, i), \ldots, a_{ni} = f(n, i)$ and $b_{i1} = f(i, 1), \ldots, b_{in} = f(i, n)$, along with the corresponding signatures $\sigma_D^{a_{ji}} := \mathsf{Sign}_{\mathsf{sk}_D}(a_{ji}, j, i)$ and $\sigma_D^{b_{ij}} := \mathsf{Sign}_{\mathsf{sk}_D}(b_{ij}, i, j)$.

Distribution check:

2: Party P_i checks whether it has received t-consistent shares $a_{1i}, \ldots, a_{ni}$ and $b_{i1}, \ldots, b_{in}$ with signatures $\sigma_D^{a_{ji}}$, $\sigma_D^{b_{ij}}$ from D. If this is the case, P_i sends $\texttt{ok}$ via $\mathsf{GBC}_{\texttt{auth}}$. Otherwise, party P_i sends $\texttt{missing}$ via $\mathsf{GBC}_{\texttt{auth}}$ and sets $\texttt{happy}_i := \texttt{false}$.

3: For every party P_i, D checks whether it has received $\texttt{ok}$ with grade 2 via $\mathsf{GBC}_{\texttt{auth}}$ from P_i in Step 2. If this is not the case, D sends the shares for P_i, i.e. $a_{1i}, \ldots, a_{ni}$ and $b_{i1}, \ldots, b_{in}$ along with the corresponding signatures $\sigma_D^{a_{ji}}$, $\sigma_D^{b_{ij}}$, to all parties via $\mathsf{GBC}_{\texttt{auth}}$.

4: For every party P_j that P_i has not received $\texttt{ok}$ with grade at least 1 via $\mathsf{GBC}_{\texttt{auth}}$ in Step 2, P_i checks whether D has responded in Step 3. Unless P_i has the t-consistent shares with signatures for P_j with grade 2 from D via $\mathsf{GBC}_{\texttt{auth}}$ in Step 3, P_i sets $\texttt{happy}_i = \texttt{false}$.
If P_i has not sent an $\texttt{ok}$ message in Step 2, it checks whether it has received its t-consistent shares $a_{1i}, \ldots, a_{ni}$ and $b_{i1}, \ldots, b_{in}$ with signatures $\sigma_D^{a_{ji}}$, $\sigma_D^{b_{ij}}$ with grade at least 1 from D via $\mathsf{GBC}_{\texttt{auth}}$ in Step 3. If this is the case, P_i uses these shares from this point on.

Pair-wise consistency checks:

5: For every party P_j, party P_i sends to P_j its share a_{ji} along with D's signature $\sigma_D^{a_{ji}}$ and its own signature $\sigma_{P_i}^{a_{ji}} := \mathsf{Sign}_{\mathsf{sk}_{P_i}}(a_{ji}, j, i)$.

6: Party P_j checks the shares received in Step 5. For every party P_i, unless P_j has received a_{ji} with signatures $\sigma_D^{a_{ji}}$, $\sigma_{P_i}^{a_{ji}}$ from P_i such that $a_{ji} = b_{ji}$, P_j sends (b_{ji}, j, i) along with D's signature $\sigma_D^{b_{ji}}$ and its own signature $\sigma_{P_j}^{b_{ji}} := \mathsf{Sign}_{\mathsf{sk}_{P_j}}(b_{ji}, j, i)$ to all parties via $\mathsf{GBC}_{\texttt{auth}}$.

7: For every party P_j, party P_i checks whether it has received (b_{ji}, j, i) with signatures $\sigma_D^{b_{ji}}$, $\sigma_{P_j}^{b_{ji}}$ with grade at least 1 via $\mathsf{GBC}_{\texttt{auth}}$ in Step 6. If this is the case, P_i sends (a_{ji}, j, i) along with the dealer's signature $\sigma_D^{a_{ji}}$ and its own signature $\sigma_{P_i}^{a_{ji}}$ to all parties via $\mathsf{GBC}_{\texttt{auth}}$.

Determine share:

8: Party P_i checks if it has observed any conflicts in Step 6 and Step 7: if it has received (a_{kj}, k, j) with D's signature $\sigma_D^{a_{kj}}$ with grade at least 1 from P_j via $\mathsf{GBC}_{\texttt{auth}}$ in Step 6, and (b_{kj}, k, j) with D's signature $\sigma_D^{b_{kj}}$ with grade at least 1 from P_k via $\mathsf{GBC}_{\texttt{auth}}$ in Step 7 such that $a_{kj} \neq b_{kj}$. If this is the case, P_i sets $\texttt{happy}_i := \texttt{false}$.

9: Party P_i sets $s_i := \perp$ if it has never received t-consistent correctly-signed shares from D. Otherwise, it sets s_i as the list of values b_{ij} with $j \in [n]$ along with D's signatures $\sigma_D^{b_{ij}}$, P_i's signatures $\sigma_{P_i}^{b_{ij}}$, and the signature $\sigma_{P_j}^{a_{ij}} = \sigma_{P_j}^{b_{ij}}$ received from each P_j (if any). Then, P_i outputs s_i and $\texttt{happy}_i$.

Protocol: $\texttt{cVSS}_{\texttt{auth}}.\texttt{Rec}$

1: Party P_i sends its shares s_i to all parties via $\mathsf{GBC}_{\texttt{auth}}$ – this is the list of values b_{ij} with $j \in [n]$ along with corresponding signatures from D, P_i, and, for every party P_j, P_j's signature for b_{ij} (if received).
2: If P_i has received the shares of P_j via $\mathsf{GBC}_{\texttt{auth}}$ with grade 2 in the previous step, it checks whether:
- P_j's claimed shares are t-consistent.
- For every b_{jk}: P_j has provided its own signature, D's signature and either: the signature from P_k, or in $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$, P_i has received P_j's signed share with grade 2 in Step 6, and P_i has not observed an appropriate response from P_k with grade 2 in Step 7.

Unless both of these conditions hold, P_i ignores the shares received from P_j.
3: Party P_i interpolates a polynomial $f_i(x, y)$ of degree at most t in each variable from the undiscarded shares and outputs $y_i := f_i(0, 0)$.

Analysis. We split the proof of Theorem 5 into a sequence of lemmas. We first note that the Syntax property follows directly from the protocol's description. In the following, Lemma 6 implies Sharing-Validity. Afterwards, Lemma 7 proves Cheating-Prevention. We then focus on Secrecy in Lemma 8, and on Correctness in Lemma 9. Finally, we prove Conditional Binding in Lemma 11. Regarding the round complexities, if R_{GBC} denotes the round complexity of the GBC protocol assumed, we note that the Sharing phase takes $R_{\texttt{Sh}} = 2 + 4 \cdot R$ rounds, while the Reconstruction phase takes $R_{\texttt{Rec}} = R$ rounds, where R denotes the round complexity of the assumed Graded Broadcast protocol $\mathsf{GBC}_{\texttt{auth}}$. As $R = 3$ by [26, 38], we obtain that $R_{\texttt{Sh}} = 14$ and $R_{\texttt{Rec}} = 3$.

Lemma 6. *Assume D is honest and let f denote the polynomial it chose in Step 1 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$. Then, every honest party P_i outputs $s_i \neq \perp$ and $\texttt{happy}_i := \texttt{true}$, where s_i contains the list $b_{ij} = f(i, j)$ with $j \in [n]$.*

Proof. As D is honest, every party P_i receives its shares $a_{ji} = f(j, i), b_{ij} = f(i, j)$ with $j \in [n]$. These shares are t-consistent and also pair-wise consistent. Then, in Step 9 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$, every honest party P_i defines s_i as the list $b_{ij} = f(i, j)$ with $j \in [n]$ along with the corresponding signatures, which completes the proof for the first part of the lemma's statement. It remains to show that every honest party P_i completes the protocol with $\texttt{happy}_i = \texttt{true}$. To do so, we analyze each of the steps of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$ where P_i may update its $\texttt{happy}_i$ flag.

In Step 2, every honest party P_i maintains $\texttt{happy}_i = \texttt{true}$ and sends $\texttt{ok}$ via $\mathsf{GBC}_{\texttt{auth}}$. These messages are received with grade 2 by all parties.

Then, in Step 4, P_i checks whether D has responded correctly and with grade 2 to every party P_j that P_i has received a `missing` message from with grade at least 1. The Graded Consistency property of $\mathsf{GBC}_{\mathtt{auth}}$ guarantees that, if P_i receives $(\mathtt{missing}, g_i)$ from P_j with $g_i \geq 1$, then D either receives $(\mathtt{missing}, g_D)$, or $(\perp, 0)$. That is, D has not received `ok` with grade 2 from P_j. In Step 3, D sends the t-consistent signed shares corresponding to P_j. Hence, P_i receives the response corresponding to any such P_j with grade 2 and maintains $\mathtt{happy}_i = \mathtt{true}$.

It remains to discuss Step 8: P_i checks whether, in Steps 5-7, it has observed shares (a_{kj}, k, j) and (b_{kj}, k, j) with signatures from D such that $a_{jk} \neq b_{kj}$. Since D is honest, it has provided P_j and P_k with shares a_{kj}, b_{kj} such that $a_{jk} = b_{kj}$. Consequently, at least one of P_j and P_k is byzantine. However, a byzantine party cannot send a different share with a valid signature from D, as it would have to forge D's signature. Consequently, P_i has not observed any conflicting shares in Step 8, and therefore completes $\mathtt{cVSS}_{\mathtt{auth}}.\mathtt{Sh}$ with $\mathtt{happy}_i = \mathtt{true}$.

Lemma 7. *$\mathtt{cVSS}_{\mathtt{auth}}$ satisfies Cheating-Prevention.*

Proof. We assume that an honest party P_i has obtained $s_i = \perp$ in $\mathtt{cVSS}_{\mathtt{auth}}.\mathtt{Sh}$, while an honest party P_j has completed $\mathtt{cVSS}_{\mathtt{auth}}.\mathtt{Sh}$ with $\mathtt{happy}_j = \mathtt{true}$.

P_i has not received signed t-consistent shares from D in Step 2, and has announced this by sending `missing` to all parties via GBC in Step 2. Every party, including P_j, receives this message with grade 2. Moreover, P_i has not received signed t-consistent shares from D with grade at least 1 in Step 4 either: otherwise, P_i would have obtained $s_i \neq \perp$. However, P_j has not set $\mathtt{happy}_j = \mathtt{false}$ in this step, and therefore it has received signed t-consistent shares for P_i from D with grade 2. We have obtained a contradiction: the Graded Consistency property of GBC ensures that P_i has received, in fact, these shares with grade at least 1, and therefore has completed $\mathtt{cVSS}_{\mathtt{auth}}.\mathtt{Sh}$ with $s_i \neq \perp$.

Lemma 8. *$\mathtt{cVSS}_{\mathtt{auth}}$ achieves Secrecy.*

Proof. We assume that D is honest with input secret s, and we show that the joint view of the byzantine parties before $\mathtt{cVSS}_{\mathtt{auth}}.\mathtt{Rec}$ starts is independent of s.

As D is honest, if f denotes the polynomial that D has chosen in Step 1 of $\mathtt{cVSS}.\mathtt{Sh}$, every party P_i receives its signed shares $a_{ji} = f(j, i)$ and $b_{ij} = f(i, j)$ in Step 1. At this point, the byzantine parties' view contains the evaluations $f(i, j)$ where P_i or P_j is byzantine, hence at most t points on any row $f(i, y)$ and at most t points on any column $f(x, j)$. Since f has degree at most t in each variable, t points along any row or column are insufficient to interpolate the corresponding univariate polynomial. These evaluations impose no restriction on the possible value of $f(0, 0) = s$: for every s' there is a polynomial f' consistent with the byzantine parties' view such that $f'(0, 0) = s'$. Consequently, the byzantine parties' joint view after Step 1 is independent of D's secret s. In the following, we show that the further steps of $\mathtt{cVSS}.\mathtt{Sh}$ where shares are disclosed do not reveal any additional information.

Since D is honest, every honest party sends `ok` in Step 2. Then, in Step 3, D publishes the shares of parties it has not received `ok` with grade 2 from, hence the shares of byzantine parties only, which reveal no new information.

Afterwards, in Step 5, every byzantine party P_j receives a_{ij} from every honest party P_i. Since D is honest, $a_{ji} = f(j,i) = b_{ji}$, which P_j has received from D in Step 1. Hence, no additional information is revealed.

In Step 6, an honest party P_j reveals b_{ji} only if it has not received $a_{ji} = b_{ji}$ in Step 5 from P_j. Since D is honest, P_j has received $b_{ji} = f(j,i) = a_{ji}$ from D in Step 1, and therefore, in this case, P_j is byzantine. Hence, the share b_{ji} was included in the byzantine parties' joint view since Step 1.

Similarly, in Step 7, an honest party P_i reveals a_{ji} only if P_j has revealed b_{ji} in Step 6. Since P_i is honest and D is honest, P_i has sent $a_{ji} = b_{ji} = f(j,i)$ to P_j in Step 5. Therefore P_j is byzantine, and no new information was revealed.

We may therefore conclude that the byzantine parties' joint view before $\texttt{cVSS}_{\texttt{auth}}.\texttt{Rec}$ is independent of the honest dealer's secret s.

Lemma 9. *$\texttt{cVSS}_{\texttt{auth}}$ achieves Correctness.*

Proof. We assume that D is honest with input s, and we show that each honest party P_i completes $\texttt{cVSS.Rec}$ with output $y_i = s$. Since D is honest, Lemma 6 guarantees that the every honest party P_i completes $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$ with shares $b_{ij} = f(i,j)$ with $j \in [n]$, where f denotes the polynomial chosen by D in Step 1. In the following, we analyze the shares that each honest party P_i considers in $\texttt{cVSS}_{\texttt{auth}}.\texttt{Rec}$.

We note that P_i receives shares from every honest party P_j with grade 2, and we show that P_i does not discard the shares received from P_j. P_i receives from party P_j the shares $b_{jk} = f(j,k)$ for $k \in [n]$, along with a signature from D (since D is honest) and a signature from P_j. If P_j has received P_k's signature $\sigma_{P_k}^{a_{jk}} = \sigma_{P_k}^{b_{jk}}$ for $a_{jk} = b_{jk}$ in Steps 5 or 7 of $\texttt{cVSS.Sh}$, then this signature is attached as well. Otherwise, P_j has sent (b_{jk}, j, k) along with D's signature and its own signature via $\mathsf{GBC}_{\texttt{auth}}$ in Step 6 of $\texttt{cVSS.Sh}$, and P_i has received this message with grade 2. If P_i has received an appropriate response from P_k to P_j with grade 2 via $\mathsf{GBC}_{\texttt{auth}}$ in Step 7 of $\texttt{cVSS.Sh}$, then P_i has received this response as well with grade 1, and therefore has attached P_k's signature to the share received by P_i in $\texttt{cVSS}_{\texttt{Rec}}$. Hence, if P_j does not attach P_k's signature, P_i observes P_j's signed share b_{jk} in Step 6 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$ and does not observe an appropriate response from P_k in Step 7 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$. Moreover, P_j's shares are t-consistent, hence P_i does not discard the shares received from P_j.

Next we show that, if P_i does not discard the shares received from a byzantine party P_j, then $b_{jk} = f(j,k)$ for every party P_k: P_i does not discard the shares received from P_j if it has received D's signature for each of these shares. Then, it must be that $b_{jk} = f(j,k)$, as the byzantine parties cannot forge D's signature.

Then, P_i defines $f_i(x,y)$ as a polynomial of degree at most t in each variable such that $f_i(j,k) = b_{jk} = f(j,k)$ for each undiscarded share b_{jk}. These are at least $(t+1)\cdot(t+1)$ (coming from the shares of the $n - t \geq t+1$ honest parties). Then, since $f(x,y)$ is also a polynomial of degree at most t in each variable, $f_i(x,y) = f(x,y)$, and thus P_i outputs $y_i = f_i(x,y) = f(x,y)$.

The next lemma will be useful in proving the Conditional Binding property.

Lemma 10. *If an honest party P_i completes $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$ with $\texttt{happy}_i = \texttt{true}$, then every honest party P_j has obtained t-consistent shares a_{kj}, b_{jk} with $k \in [n]$. Moreover, the honest parties' shares are pair-wise consistent: if P_j and P_k are two honest parties, then $a_{kj} = b_{kj}$.*

Proof. We first note that P_i has not set $\texttt{happy}_i = \texttt{false}$ in Step 4. Assuming that an honest party P_j has not received t-consistent shares from D, P_j has sent a `missing` message in Step 2, which P_i receives with grade 2. In Step 4, P_i has received D's response to P_j with grade 2, and this response consists of signed t-consistent shares. As P_i has received this response with grade 2, $\mathsf{GBC}_{\texttt{auth}}$ ensures that P_j has received the same response with grade at least 1, and therefore now holds t-consistent shares.

It remains to show that the honest parties' shares are pair-wise consistent. As P_i has not set $\texttt{happy}_i = \texttt{false}$ in Step 8, it has not observed any pair of shares $(a_{kj}, k, j), (b_{kj}, k, j)$ signed by D such that $a_{kj} \neq b_{kj}$. Assuming that two honest parties P_i and P_j hold $a_{kj} \neq b_{kj}$, P_k would have announced this by sending (b_{kj}, k, j) along with D's signature in Step 6. Then, P_j would have responded by sending (a_{kj}, k, j) along with D's signature in Step 7. P_i would have received these messages with grade 2, and would have set $\texttt{happy}_i = \texttt{false}$ in Step 8. Therefore, honest parties' shares are pair-wise consistent.

Lemma 11. *$\texttt{cVSS}_{\texttt{auth}}$ achieves Conditional Binding.*

Proof. We assume that at least one honest party P_i holds $\texttt{happy}_i = \texttt{true}$, and we show that the joint view of the honest parties at the end of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$ defines a value y' such that all honest parties output y' in $\texttt{cVSS}_{\texttt{auth}}.\texttt{Rec}$. In the following, we first show that the honest parties' shares at the end of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$ lie on a unique polynomial $f'(x, y)$ of degree at most t in each variable. Afterwards, we show that, in $\texttt{cVSS}_{\texttt{auth}}.\texttt{Rec}$, every honest party outputs $y_i = f(0, 0)$.

By Lemma 10, since P_i has completed $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$ with $\texttt{happy}_i = \texttt{true}$, the honest parties have obtained t-consistent shares that are also pair-wise consistent. We may then consider the set $\mathcal{H}$ containing the $t+1$ honest parties with the lowest indices, and define a polynomial $f'(x, y)$ of degree at most t in each variable based on their consistency cross-checks: $f'(k, j) = a_{kj} = b_{kj}$ for every P_j, P_k in $\mathcal{H}$. These are $(t+1) \cdot (t+1)$ evaluations, and, as a bi-variate polynomial of degree at most t in each variable is uniquely defined by such constraints, they uniquely determine $f'(x, y)$. In the following, we show that all of the honest shares lie on the polynomial f'.

We first consider the shares b_{jk} with $k \in [n]$ of a party $P_j \in \mathcal{H}$. Recall that the shares b_{jk} with $k \in [n]$ of P_j are t-consistent, hence they lie on a unique polynomial $B(x)$ of degree at most t. By construction of f', $f'(j, k) = b_{jk} = B(k)$ for every $P_k \in \mathcal{H}$. The univariate polynomial $f'(j, y)$ also has degree at most t, and $B(x)$ and $f'(j, y)$ agree in $t + 1$ points. Hence they are identical, and $b_{jk} = B(k) = f'(j, k)$ for every $k \in [n]$.

Next, we note that the shares a_{kj} with $k \in [n]$ of any party $P_j \in \mathcal{H}$ also lie on f'. We recall that the shares a_{kj} with $k \in [n]$ are t-consistent, hence they lie on a unique polynomial $A(x)$ of degree at most t: $A(k) = a_{kj}$ for every $k \in [n]$.

In addition, $a_{kj} = b_{kj} = f'(k, j)$ for every $P_k \in \mathcal{H}$. The univariate polynomial $f'(x, j)$ also has degree at most t, and $A(x)$ and $f'(x, j)$ agree in $t + 1$ points. Hence, they are identical: $a_{kj} = A(k) = f'(k, k)$ for every $k \in [n]$.

It remains to consider the shares b_{jk} with $k \in [n]$ of honest parties $P_j \notin \mathcal{H}$. These shares are also t-consistent: they lie on a unique polynomial $B(x)$ of degree at most t: $B(k) = b_{jk}$. P_j has obtained $b_{jk} = a_{jk} = f'(j, k)$ for every $P_k \in \mathcal{H}$. Hence $B(x)$ and $f'(j, y)$ agree in at least $t + 1$ points. Since $f'(j, y)$ is also a univariate polynomial of degree at most t, $f'(j, y)$ and $B(x)$ are identical. Hence, $b_{jk} = B(k) = f'(j, k)$ for every $k \in [n]$.

Consequently, the shares of all honest parties lie on a well-defined polynomial $f'(x, y)$ of degree at most t in each variable. We may now analyze $\texttt{cVSS}_{\texttt{auth}}.\texttt{Rec}$ and show that every honest party P_l obtains $f_l(x, y) = f'(x, y)$ and therefore outputs $y_l = f_l(0, 0) = f'(0, 0)$. Every honest party P_l receives the shares b_{jk} with $k \in [n]$ of each honest party P_j, along with corresponding signatures: D's signature of D, P_j's signature, and possibly P_k's signature . If P_j has received the signature of P_k in Step 5 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$, this signature is attached. Otherwise, P_j has sent (b_{jk}, j, k) along with D's signature via $\mathsf{GBC}_{\texttt{auth}}$ in Step 6 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$, and P_i has received this message with grade 2. If P_l has observed an appropriate response to P_j from P_k with grade 2 in Step 7 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$, then P_j has received this response as well with grade at least 1, thus has attached P_k's signature to its share in $\texttt{cVSS}_{\texttt{auth}}.\texttt{Rec}$. Moreover, the shares b_{jk} with $k \in [n]$ are t-consistent: otherwise, no honest party P_i would hold $\texttt{happy}_i = \texttt{true}$. Hence, P_l does not discard the shares of P_j. Then, it must be that $f_l(j, k) = b_{j,k} = f'(j, k)$ for every honest party P_j, and for every party P_k.

P_l additionally receives shares from up to t byzantine parties. We show that, if P_l does not discard the shares of a byzantine party P_j, then these must also lie on f'. P_l receives the shares b_{jk} with $k \in [n]$ from party P_j, along with corresponding signatures: D's signature, P_j's signature, and possibly P_k's signature. The shares b_{jk} with $k \in [n]$ are t-consistent, hence they lie on a polynomial $B(x)$ degree at most t – otherwise, P_l discards these shares. We then analyze the shares b_{jk} where P_k is honest. If P_j has attached P_k's signature on this share, then $b_{jk} = a_{jk} = f'(j, k)$. Otherwise, since P_l does not discard the share of P_j, P_l has received from P_j the share (b_{jk}, j, k) along with D's signature with grade 2 via $\mathsf{GBC}_{\texttt{auth}}$ in Step 6 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$, and a response with grade 2 from P_k in Step 7. In Step 8 of $\texttt{cVSS}_{\texttt{auth}}.\texttt{Sh}$, P_i, holding $\texttt{happy}_i := \texttt{true}$, has received these messages from P_j with grade at least 1, and from P_k with grade 2, and has observed that $b_{jk} = a_{jk}$. P_l has also observed the response of P_k with grade 2 in Step 7: hence, in this case P_l discards the shares of P_j. We have therefore obtained that, if P_l does not discard the shares of P_j, then $b_{jk} = B(k) = f'(j, k)$ for the $n - t \geq t + 1$ honest parties P_k. As both $B(x)$ and $f'(j, x)$ are univariate polynomials of degree at most t, $B(x)$ and $f'(j, x)$ must be identical, and $b_{jk} = f'(j, k)$ for every $k \in [n]$.

We have obtained that, if P_l does not discard the shares of P_j, it must be that $b_{jk} = f'(j, k)$ for every $k \in [n]$. As P_l considers the shares of at least $n - t \geq t + 1$ parties, these uniquely determine a polynomial $f_l(x, y)$ of degree at most t in each variable. Then, since both $f_l(x, y)$ and $f'(x, y)$ are polynomials of degree at

most t in each variable that agree in at least $(t+1)\cdot(t+1)$ points, it must be that $f_l(x,y) = f'(x,y)$. Consequently, P_l outputs $y_l = f_l(0,0) = f'(0,0)$.

6 ProxCoin

We now introduce the construction of our (ℓ,δ)-ProxCoin protocol `ProxCoin`. The protocol uses as building blocks: (i) an $\ell_{\texttt{Prox}}$-Proxcensus protocol `Prox` secure up to t corruptions with $R_{\texttt{Prox}}$ rounds, and (ii) a `cVSS` protocol secure up to t corruptions with $R_{\texttt{Sh}}$ for sharing and $R_{\texttt{Rec}}$ for reconstruction. We first state the theorem achieved from the two building blocks.

Theorem 6. *Let $t < n$, $\ell \geq 2$ and $\ell_{Prox} < \ell$. Further let `Prox` and `cVSS` denote the building blocks as described above. Then, `ProxCoin` is a (ℓ,δ)-ProxCoin protocol with $\delta = \lceil t\cdot(\ell-1)/(\ell_{Prox}-1)\rceil$ secure up to t corruptions with round complexity $R_{ProxCoin} := R_{Prox} + R_{Sh} + R_{Rec}$.*

By instantiating the underlying building blocks `Prox` and `cVSS`, we obtain `ProxCoin` protocols secure against $t < n/3$ corruptions in the plain model, and against $t < n/2$ corruptions assuming a bulletin-board PKI. The first corollary is achieved by instantiating `Prox` with the protocol in Theorem 9, described in Appendix A. and `cVSS` with that of Theorem 4.

Corollary 1. *Let $L \geq \frac{2t}{n-2t}$ and $\ell = \lfloor \frac{1}{2}\left(\frac{n-2t}{t}\right)^L L^L \rfloor$ be natural numbers. Then, `ProxCoin` is a (ℓ,δ)-ProxCoin protocol with $\delta := t$, secure up to $t < n/3$ corruptions with round complexity $R_{ProxCoin} = 3\cdot L + 11$.*

The second corollary is achieved by instantiating `Prox` with the protocol introduced in [32] and `cVSS` with the protocol described by Theorem 5.

Corollary 2. *Consider a constant $\varepsilon > 0$. Let $L \geq \frac{1}{\varepsilon} - 1$, $\ell = \lfloor \frac{1}{2}\left(\frac{2\varepsilon}{1-\varepsilon}\right)^L L^L \rfloor$ be natural numbers. Then, `ProxCoin` is (ℓ,δ)-ProxCoin protocol with $\delta := t$, secure up to $t = (1-\varepsilon)n/2$ corruptions with round complexity $R_{ProxCoin} := 3\cdot L + 17$.*

Protocol Description. We describe the protocol below. Each party shares a uniform random value using `cVSS`, and then, following the approach of [28], the final coin will be computed as a *weighted sum* of the reconstructed shares. The weights satisfy a few properties : (i) the weights w^j computed for each corrupted P_j differ by at most $1/(\ell_{\texttt{Prox}}-1)$; (ii) if the share of a corrupted party P_j cannot be reconstructed, every honest party obtains $w^j = 0$, and (iii) every honest party obtains $w^j = 1$ for every honest party P_j. These properties ensure that the honest parties' coins are well-defined, and differ by at most $\delta := \lceil t\cdot(\ell-1)/(\ell_{\texttt{Prox}}-1)\rceil$.

Protocol: `ProxCoin`

Building Blocks: An $\ell_{\texttt{Prox}}$-Proxcensus `Prox`, and `cVSS`.

Parameters: ℓ number of slots and $\delta = \lceil t\cdot(\ell-1)/(\ell_{\texttt{Prox}}-1)\rceil$ the difference of

number of slots between honest parties.

Player P_i does the following:

1: Choose x_i uniformly at random from $\{0, \ldots, \ell - 1\}$.
2: Distribute the secret x_i to all parties via $\texttt{cVSS.Sh}$.
3: Denote the outputs of the $\texttt{cVSS.Sh}$ invocation having P_j as dealer by $s_i^j, \texttt{happy}_i^j$.
4: For every party P_j, join $\texttt{Prox}$ with input $\texttt{happy}_i^j$. Upon obtaining output z_i^j, set $w_i^j := z_i^j/(\ell_{\texttt{Prox}} - 1)$.
5: For every party P_j, join the $\texttt{cVSS.Rec}$ invocation having P_j as dealer with input s_i^j and let x_i^j denote the secret obtained. If $x_i^j = \perp$, set $x_i^j := 0$.
6: Output $\left\lceil \sum_{P_j} w_i^j \cdot x_i^j \right\rceil \mod \ell$.

Analysis. We split the proof of Theorem 6 into multiple lemmas: Lemma 15 ensures that One Uniform Randomness holds, and Lemma 16 discusses Consistency, establishing that the $\texttt{ProxCoin}$ requirements are satisfied. The round complexity in Theorem 6 follows from the description. We first discuss some properties.

Lemma 12. *If P_j is honest, every honest party P_i obtains $w_i^j := 1$ and $x_i^j = x_j$.*

Proof. By the Sharing-Validity property of $\texttt{cVSS}$, every party P_i holds $\texttt{happy}_i^j = \texttt{true}$ for every honest party P_j at the end of the $\texttt{cVSS.Sh}$ invocation with dealer P_j. Then, the Validity property of $\texttt{Prox}$ then ensures that every party obtains $s_i^j = \ell_{\texttt{Prox}} - 1$, and therefore $w_i^j = 1$. In addition, $\texttt{cVSS}$'s Correctness ensures that, at the end of the $\texttt{cVSS.Rec}$ invocation having P_j as dealer, P_i obtains $x_i^j = x_j$.

Lemma 13. *If an honest party P_i obtains $x_i^k := \perp$ for some party P_k, then P_k is corrupted and every honest party P_j obtains $w_j^k := 0$.*

Proof. Due to the Conditional Binding property of $\texttt{cVSS}$, if P_i obtains $x_i^k := \perp$, every honest party P_j holds $\texttt{happy}_j^k = \texttt{false}$ at the end of the $\texttt{cVSS.Sh}$ invocation having P_k as a dealer. Therefore, all honest parties join $\texttt{Prox}$ with input 0, and the Validity guarantee of $\texttt{Prox}$ ensures that every honest party P_j obtains $s_j^k = 0$, hence sets $w_j^k := 0$. Moreover, the Sharing-Validity property of $\texttt{cVSS}$ ensures that if some honest party P_j obtains $\texttt{happy}_j^k := \texttt{false}$, then P_k is corrupted.

Lemma 14. *Assume an honest party P_i obtains $w_i^l > 0$ for some party P_l. Then, there is a value x such that every honest party P_j obtains $x_j^l = x$. Moreover, if both P_j and P_k are honest, $\left|w_j^l - w_k^l\right| \leq 1/(\ell_{\texttt{Prox}-1})$.*

Proof. P_i has obtained $w_i^l > 0$, and therefore $s_i^l > 0$. $\texttt{Prox}$'s Validity property ensures that there is some honest party P_r that has obtained $\texttt{happy}_r^l = \texttt{true}$ in the $\texttt{cVSS.Sh}$ invocation with dealer P_l. Then, $\texttt{cVSS}$'s Conditional Binding ensures that there is a value x uniquely defined by the honest parties s^l such that every honest party P_j obtains $x_j^l = x$. Afterwards, $\texttt{Prox}$'s Consistency ensures that, if P_j and P_k are honest, they obtain slots $s_j^l, s_k^l \in [0, \ell_{\texttt{Prox}} - 1]$, such that $\left|s_j^l - s_k^l\right| \leq 1$ and consequently $\left|w_j^l - w_k^l\right| \leq 1/(\ell_{\texttt{Prox}-1})$.

Lemma 15. *`ProxCoin` achieves One Uniform Randomness.*

Proof. As established by Lemma 12, every honest party P_i assigns weight $w_i^j = 1$ and $x_i^j = x_j$ for every honest party P_j. Since honest parties have chosen their secrets x uniformly at random from $\mathbb{Z}_\ell$, the sum $\sum_{\text{honest } P_j} w_i^j \cdot x_i^j \mod \ell$ is also uniformly distributed over $\mathbb{Z}_\ell$.

By Lemma 13, if an honest party P_i obtains $x_i^k = \perp$ at the end of the `cVSS.Rec` invocation having P_k as a dealer, then P_k is corrupted and every honest party P_j holds $w_j^k = 0$. Note that the summands corresponding to such corrupted parties P_k are not included in the overall weighted sum. Moreover, note that $x_i^k = \perp$ implies that all honest parties obtained $\texttt{happy}^k = \texttt{false}$ at the end of `cVSS.Sh`, so this value is chosen independently of the honest parties' secrets.

Lastly, we consider the corrupted parties P_k such that at least one honest party P_i holds $\texttt{happy}_k^i = \texttt{true}$. In this case, `cVSS`'s Conditional Binding ensures that the honest parties obtain the same value x_i^k at the end of the Reconstruction phase, and this value was chosen independently of the honest parties' secrets x_i.

Note that for these parties, the corrupted parties can collude to shift the weights of honest parties. This is however not a problem, as the random values of the honest parties are only revealed once the weights for all parties are set.

As $\sum_{\text{honest } P_j} w_i^j \cdot x_i^j \mod \ell$ is uniformly distributed over $\mathbb{Z}_\ell$, the claim follows.

Lemma 16. *`ProxCoin` achieves Consistency for $\delta := \lceil t \cdot (\ell - 1)/(\ell_{\texttt{Prox}} - 1) \rceil$.*

Proof. Let c_i and c_j denote the outputs of two honest parties P_i and P_j. We show that $d_\ell(c_i, c_j) \leq \lceil t \cdot (\ell - 1)/(\ell_{\texttt{Prox}} - 1) \rceil$. We note that $d_\ell(x, y) = \min\{|x - y|, \ell - |x - y|\} \leq |x - y|$, which enables us to upper bound for $d_\ell(c_i, c_j)$ as follows:

$$\begin{aligned}
d_\ell(c_i, c_j) &= d_\ell\left(\left\lceil \sum_{P_k} w_i^k \cdot x_i^k \right\rceil \mod \ell, \left\lceil \sum_{P_k} w_j^k \cdot x_j^k \right\rceil \mod \ell\right) \\
&\leq \left| \left\lceil \sum_{P_k} w_i^k \cdot x_i^k \right\rceil - \left\lceil \sum_{P_k} w_j^k \cdot x_j^k \right\rceil \right| \\
&\leq \left| \left\lceil \sum_{P_k} w_i^k \cdot x_i^k - \sum_{P_k} w_j^k \cdot x_j^k \right\rceil \right| \\
&= \left| \left\lceil \sum_{P_k} w_i^k \cdot x_i^k - w_j^k \cdot x_j^k \right\rceil \right| \\
&\leq \left\lceil \sum_{P_k} \left| w_i^k \cdot x_i^k - w_j^k \cdot x_j^k \right| \right\rceil .
\end{aligned}$$

We partition the set of parties into three sets: $\mathcal{H}$, denoting the set of honest parties, $\mathcal{B}_0$, denoting the set of corrupted parties P_k such that $w_i^k = w_j^k = 0$, and $\mathcal{B}_w$, denoting the set of corrupted parties P_k such that $w_i^k > 0$ or $w_j^k > 0$ We

may then write $\sum_{P_k}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right|$ as follows, which enables us to provide an upper bound by analyzing the sums over $\mathcal{H}$, $\mathcal{B}_0$, and $\mathcal{B}_w$ separately:

$$\sum_{P_k}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| = \sum_{P_k \in \mathcal{H}}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| + \sum_{P_k \in \mathcal{B}_0}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| + \sum_{P_k \in \mathcal{B}_w}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right|.$$

We first consider the sum over $\mathcal{H}$: Lemma 12 ensures that, for every honest party P_k, P_i and P_j have obtained weights w_i^k and w_k^j such that $w_i^k = w_j^k = 1$, and secrets x_i^k and x_j^k such that $x_i^k = x_j^k$. This implies that the weighted sums over the honest parties' values are equal, i.e. $\sum_{P_k \in \mathcal{H}}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| = 0$. For the sum over $\mathcal{B}_0$, we obtain $\sum_{P_k \in \mathcal{B}_0}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| = 0$ since $w_i^k = w_j^k = 0$ for every party $P_k \in \mathcal{B}_0$. It remains to discuss the sum over $\mathcal{B}_w$. In this case, Lemma 14 ensures that, for every party $P_k \in \mathcal{B}_w$, P_i and P_j have obtained weights w_i^k and w_j^k satisfying $\left|w_i^k - w_j^k\right| \leq 1/(\ell_{\texttt{Prox}} - 1)$ and have reconstructed $x_i^k = x_j^k = x$. This implies that, $\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| = \left|(w_i^k - w_j^k) \cdot x\right| \leq (\ell - 1)/(\ell_{\texttt{Prox}} - 1)$. Moreover, we note that $\left|\mathcal{B}_w\right| \leq t$, which enables us to conclude that $\sum_{P_k \in \mathcal{B}_w}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| \leq t \cdot (\ell - 1)/(\ell_{\texttt{Prox}} - 1)$.

Consequently, we have shown that $\sum_{P_k}\left|w_i^k \cdot x_i^k - w_j^k \cdot x_j^k\right| \leq t \cdot (\ell - 1)/(\ell_{\texttt{Prox}} - 1)$, and hence $d_\ell(c_i, c_j) \leq \lceil t \cdot (\ell - 1)/(\ell_{\texttt{Prox}} - 1)\rceil$. We may therefore conclude that δ-Consistency holds for $\delta = \lceil t \cdot (\ell - 1)/(\ell_{\texttt{Prox}} - 1)\rceil$.

7 Putting it All Together

We may now combine the results presented in the previous sections and state our final theorems. We defer the complete proofs to Appendix B.

In the plain model, we may rely on the Proxcensus protocol described in Appendix A and on the ProxCoin protocol of Corollary 1. By using these protocols in $\Pi_{\texttt{EE}}^{\ell+1}$, Theorem 3 enables us to state the following result.

Theorem 7. *Let $L \geq \frac{2t}{n-2t}$ be a natural number such that $L^L > t^2$ and $\ell = \lfloor \frac{1}{2}\left(\frac{n-2t}{t}\right)^L L^L \rfloor \geq 2$. Then, if $r = L/2$, there is a $(12r + 11)$-round Byzantine Agreement protocol secure against $t < n/3$ corruptions that achieves Agreement except with probability $12 \cdot \left(2 \cdot \left(\frac{n-2t}{t}\right)^2\right)^{-r} \cdot r^{-r}$.*

In the authenticated setting, Theorem 3 provides the result below, relying on the Proxcensus protocol of [32] and on the ProxCoin protocol of Corollary 2.

Theorem 8. *Consider a constant $\varepsilon > 0$, and let $L \geq \frac{1}{\varepsilon} - 1$ be a natural number such that $L^L > t^2$ and $\ell = \lfloor \frac{1}{2}\left(\frac{2\varepsilon}{1-\varepsilon}\right)^L L^L \rfloor \geq 2$. Then, if $r = L/2$, there is a $(12r + 17)$-round authenticated Byzantine Agreement protocol secure against*

$t = (1-\varepsilon)n/2$ *corruptions that achieves Agreement except with probability* $12 \cdot \left(2 \cdot \left(\frac{2\varepsilon}{1-\varepsilon}\right)^2\right)^{-r} \cdot r^{-r}$.

References

1. Ittai Abraham, Srinivas Devadas, Danny Dolev, Kartik Nayak, and Ling Ren. Synchronous Byzantine agreement with expected $O(1)$ rounds, expected $o(n^2)$ communication, and optimal resilience. In *International Conference on Financial Cryptography and Data Security*, pages 320–334. Springer, 2019.
2. Andreea B. Alexandru, Julian Loss, Charalampos Papamanthou, Giorgos Tsimos, and Benedikt Wagner. Sublinear-round broadcast without trusted setup. In Yossi Azar and Debmalya Panigrahi:, editors, *36th SODA*, pages 4132–4171. ACM-SIAM, January 2025.
3. Hagit Attiya and Keren Censor-Hillel. Lower bounds for randomized consensus under a weak adversary. *SIAM Journal on Computing*, 39(8):3885–3904, 2010.
4. Michael Ben-Or. Another advantage of free choice: Completely asynchronous agreement protocols (extended abstract). In *Proceedings of the 2nd Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 27–30, 1983.
5. Michael Ben-Or, Danny Dolev, and Ezra N. Hoch. Brief announcement: Simple gradecast based algorithms. In Nancy A. Lynch and Alexander A. Shvartsman, editors, *Distributed Computing*, pages 194–197, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
6. Michael Ben-Or, Danny Dolev, and Ezra N. Hoch. Simple gradecast based algorithms. *CoRR*, abs/1007.1049, 2010.
7. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.
8. Elette Boyle, Ran Cohen, and Aarushi Goel. Breaking the $O(\sqrt{(}n))$-bit barrier: Byzantine agreement with polylog bits per party. In Avery Miller, Keren Censor-Hillel, and Janne H. Korhonen, editors, *40th ACM PODC*, pages 319–330. ACM, July 2021.
9. Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in Constantinople: Practical asynchronous byzantine agreement using cryptography. *Journal of Cryptology*, 18(3):219–246, July 2005.
10. Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *OSDI*, volume 99, pages 173–186, 1999.
11. T-H. Hubert Chan, Rafael Pass, and Elaine Shi. Round complexity of Byzantine agreement, revisited. Cryptology ePrint Archive, Report 2019/886, 2019. https://ia.cr/2019/886.
12. T.-H. Hubert Chan, Rafael Pass, and Elaine Shi. Sublinear-round byzantine agreement under corrupt majority. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 246–265. Springer, Cham, May 2020.
13. Jing Chen and Silvio Micali. Algorand: A secure and efficient distributed ledger. *Theoretical Computer Science*, 777:155–183, 2019.
14. Benny Chor, Michael Merritt, and David B Shmoys. Simple constant-time consensus protocols in realistic failure models. *Journal of the ACM (JACM)*, 36(3):591–614, 1989.

15. Ran Cohen, Sandro Coretti, Juan A. Garay, and Vassilis Zikas. Probabilistic termination and composability of cryptographic protocols. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 240–269. Springer, Berlin, Heidelberg, August 2016.
16. Ran Cohen, Sandro Coretti, Juan A. Garay, and Vassilis Zikas. Round-preserving parallel composition of probabilistic-termination cryptographic protocols. In Ioannis Chatzigiannakis, Piotr Indyk, Fabian Kuhn, and Anca Muscholl, editors, *ICALP 2017*, volume 80 of *LIPIcs*, pages 37:1–37:15. Schloss Dagstuhl, July 2017.
17. Ran Cohen, Iftach Haitner, Nikolaos Makriyannis, Matan Orland, and Alex Samorodnitsky. On the round complexity of randomized Byzantine agreement. In Jukka Suomela, editor, *33rd International Symposium on Distributed Computing (DISC 2019)*, volume 146 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 12:1–12:17, Dagstuhl, Germany, 2019. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.
18. Ronald Cramer, Ivan Damgård, Stefan Dziembowski, Martin Hirt, and Tal Rabin. Efficient multiparty computations secure against an adaptive adversary. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 311–326. Springer, Berlin, Heidelberg, May 1999.
19. Danny Dolev and H. Strong. Authenticated algorithms for Byzantine agreement. *SIAM J. Comput.*, 12:656–666, 11 1983.
20. Danny Dolev and H. Raymond Strong. Authenticated algorithms for Byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.
21. Danny Dolev and Andrew Yao. On the security of public key protocols. *IEEE Transactions on information theory*, 29(2):198–208, 1983.
22. Paul Feldman and Silvio Micali. Optimal algorithms for byzantine agreement. In *20th ACM STOC*, pages 148–161. ACM Press, May 1988.
23. Pesech Feldman and Silvio Micali. An optimal probabilistic protocol for synchronous Byzantine agreement. *SIAM Journal on Computing*, 26(4):873–933, 1997.
24. Michael J. Fischer and Nancy A. Lynch. A lower bound for the time to assure interactive consistency. *Information Processing Letters*, 14(4):183–186, 1982.
25. Matthias Fitzi and Juan A. Garay. Efficient player-optimal protocols for strong and differential consensus. In Elizabeth Borowsky and Sergio Rajsbaum, editors, *22nd ACM PODC*, pages 211–220. ACM, July 2003.
26. Matthias Fitzi, Chen-Da Liu-Zhang, and Julian Loss. A new way to achieve round-efficient byzantine agreement. In Avery Miller, Keren Censor-Hillel, and Janne H. Korhonen, editors, *40th ACM PODC*, pages 355–362. ACM, July 2021.
27. Matthias Fitzi and Jesper Buus Nielsen. On the number of synchronous rounds sufficient for authenticated Byzantine agreement. In *International Symposium on Distributed Computing*, pages 449–463. Springer, 2009.
28. Luciano Freitas, Petr Kuznetsov, and Andrei Tonkikh. Distributed Randomness from Approximate Agreement. In Christian Scheideler, editor, *36th International Symposium on Distributed Computing (DISC 2022)*, volume 246 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 24:1–24:21, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
29. Juan A. Garay, Jonathan Katz, Chiu-Yuen Koo, and Rafail Ostrovsky. Round complexity of authenticated broadcast with a dishonest majority. In *48th FOCS*, pages 658–668. IEEE Computer Society Press, October 2007.
30. Juan A. Garay and Yoram Moses. Fully polynomial byzantine agreement in t+1 rounds. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing (STOC)*, pages 31–41, 1993.

31. Rosario Gennaro, Yuval Ishai, Eyal Kushilevitz, and Tal Rabin. The round complexity of verifiable secret sharing and secure multicast. In *Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing*, STOC '01, page 580–589, New York, NY, USA, 2001. Association for Computing Machinery.
32. Diana Ghinea, Vipul Goyal, and Chen-Da Liu-Zhang. Round-Optimal Byzantine Agreement. In *Eurocrypt 2022, Trondheim, Norway*, May 2022.
33. A.R. Karlin and A.C. Yao. Probabilistic lower bounds for Byzantine agreement and clock synchronization. 1984.
34. Jonathan Katz and Chiu-Yuen Koo. On expected constant-round protocols for byzantine agreement. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 445–462. Springer, Berlin, Heidelberg, August 2006.
35. Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.
36. Benoît Libert, Marc Joye, and Moti Yung. Born and raised distributively: fully distributed non-interactive adaptively-secure threshold signatures with short shares. In Magnús M. Halldórsson and Shlomi Dolev, editors, *33rd ACM PODC*, pages 303–312. ACM, July 2014.
37. Yehuda Lindell, Anna Lysyanskaya, and Tal Rabin. On the composition of authenticated Byzantine agreement. *Journal of the ACM (JACM)*, 53(6):881–917, 2006.
38. Silvio Micali and Vinod Vaikuntanathan. Optimal and player-replaceable consensus with an honest majority. 2017.
39. M. Pease, R. Shostak, and L. Lamport. Reaching agreement in the presence of faults. *J. ACM*, 27(2):228–234, 4 1980.
40. Birgit Pfitzmann and Michael Waidner. *Information-theoretic pseudosignatures and Byzantine agreement for* $t \geq n/3$. IBM, 1996.
41. Michael O. Rabin. Randomized byzantine generals. In *24th FOCS*, pages 403–409. IEEE Computer Society Press, November 1983.
42. Shravan Srinivasan, Julian Loss, Giulio Malavolta, Kartik Nayak, Charalampos Papamanthou, and Sri Aravinda Krishnan Thyagarajan. Transparent batchable time-lock puzzles and applications to byzantine consensus. In *Proceedings of the 26th International Conference on the Theory and Practice of Public-Key Cryptography (PKC), part I*, pages 554–584, 2023.
43. Russell Turpin and Brian A Coan. Extending binary Byzantine agreement to multivalued Byzantine agreement. *Information Processing Letters*, 18(2):73–76, 1984.
44. Jun Wan, Hanshen Xiao, Srinivas Devadas, and Elaine Shi. Round-efficient byzantine broadcast under strongly adaptive and majority corruptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 412–456. Springer, Cham, November 2020.
45. Jun Wan, Hanshen Xiao, Elaine Shi, and Srinivas Devadas. Expected constant round byzantine broadcast under dishonest majority. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part I*, volume 12550 of *LNCS*, pages 381–411. Springer, Cham, November 2020.

Appendix

A Unauthenticated Round-Optimal Proxcensus

We describe a round-optimal Proxcensus protocol in the plain model, described by the theorem below. This is obtained by replacing one of the building blocks in the protocol of [32], which relies on digital signatures.

Theorem 9. *Let* $L \geq \frac{2t}{n-2t}$ *and* $\ell = \lfloor \frac{1}{2} \left(\frac{n-2t}{t}\right)^L L^L \rfloor$ *be natural numbers. Then, there is a* $3L$*-round* $(\ell+1)$*-Proxcensus protocol* $\texttt{Prox}_L$ *resilient against* $t < n/3$ *corruptions*

The protocol of [32], denoted by $\texttt{Prox}_{\texttt{auth}}$ operates in iterations where the parties distribute their current slot values via *Conditional Graded Broadcast*, and each party computes a new slot value by aggregating the values obtained. Conditional Graded Broadcast, defined below, achieves the same properties as Gradecast, with the addition that it explicitly enables parties to decide whether to actively participate in a party's invocation or not. If during some iteration a party receives the value of P_k with grade $g < 2$, then it marks P_k as corrupted and stops actively participating in the Conditional Graded Broadcast invocations that have P_k as sender in further iterations. Conditional Graded Broadcast enables this by having each party to join with an input bit denoting whether it will actively participate or not. It then requires that, if no honest party actively participates, every honest party outputs $(\perp, 0)$, and therefore no honest party takes into account the sender's value in the corresponding iteration of $\texttt{Prox}_{\texttt{auth}}$. This plays a crucial role in the round-optimality guarantees of $\texttt{Prox}_{\texttt{auth}}$.

Definition 6 (Conditional Graded Broadcast). *A protocol* Π *where initially a designated party* P_s *(called the sender) holds a value* x*, each party* P_i *holds a bit* b_i*, and each party* P_i *terminates upon generating an output pair* (y_i, g_i) *with* $g_i \in \{0, 1, 2\}$*, is a Conditional Graded Broadcast protocol resilient against* t *corruptions if the following properties hold whenever up to* t *parties are corrupted:*
Conditional Validity*: If* P_s *is honest and each honest party has* $b_i = 1$*, then every honest party outputs* $(x, 2)$*.*
Conditional Graded Consistency*: For any honest parties* P_i *and* P_j*:*
- $\left|g_i - g_j\right| \leq 1$*.*
- *If* $g_i > 0$ *and* $g_j > 0$*, then* $y_i = y_j$*.*

No Honest Participation*: If all honest parties input* $b_i = 0$*, then every honest party outputs* $(\perp, 0)$*.*

The Proxcensus protocol $\texttt{Prox}_{\texttt{auth}}$ of [32] operates without signatures and makes use of a Conditional Graded Broadcast (cGBC) protocol as a black box (the authors only use signatures when constructing cGBC itself). We rewrite the statement of [32, Theorem 3] to explicitly assume a Conditional Graded Broadcast protocol as follows.

Theorem 10 ((Adjusted) Theorem 3 of [32]). *Assume a Conditional Graded Broadcast protocol* cGBC *with round complexity* R_{cGBC}*, and let* $L \geq \frac{2t}{n-2t}$ *and* $\ell = \lfloor \frac{1}{2} \left(\frac{n-2t}{t}\right)^L L^L \rfloor$ *be natural numbers.*

Then, there is a $(R_{\mathsf{cGBC}} \cdot L)$*-round* $(\ell+1)$*-Proxcensus protocol with* $\ell+1$ *slots when up to* $t < n/2$ *of the parties involved are corrupted, and whenever* cGBC *achieves Conditional Graded Broadcast.*

We obtain a Conditional Graded Broadcast protocol in the plain model by making a minor adjustment to the protocol of [5] (presented in the full version [6]) which achieves Graded Broadcast secure against $t < n/3$ byzantine corruptions. Concretely, if a party P_i joins with input bit $b_i := 1$, it follows the protocol of [5]. Otherwise, it simply does not send any message, but runs the output determination step.

Protocol: cGBC

Code for sender P_s with inputs x, b_S

1: **if** $b_S = 1$ **then**
2: Round 1: Send x to all parties.
3: **end if**

Code for party P_i with input b_i

1: **if** $b_i = 1$ **then**
2: Round 2: If you have received x' from P_s by the end of round 1, send x' to all parties.
3: Round 3: Let `maj` denote the value received the most (break ties arbitrarily) in round 2, and $\#$`maj` the number of occurrences of `maj`. If $\#\texttt{maj} \geq n - t$, send `maj` to all parties.
4: **end if**

Code for party P_i: Output Determination

1: Let `maj`$'$ be the value received most often in round 3 (break ties arbitrarily), and let $\#$`maj`$'$ denote the number of occurrences of `maj`$'$.
2: If $\#\texttt{maj}' \geq n - t$, set $v :=$ `maj`$'$ and $g := 2$.
3: Otherwise, if $\#\texttt{maj}' > t$, set $v :=$ `maj`$'$ and $g := 1$.
4: Otherwise, set $v := \perp$ and $g := 0$.
5: Output (v, g).

The theorem below describes the guarantees of cGBC. Then, Theorem 9 follows directly from Theorem 10 and Theorem 11.

Theorem 11 ((Adjusted) Theorem 1 of [6]). *Protocol* cGBC *achieves Conditional Graded Broadcast within* $R_{\mathsf{cGBC}} = 3$ *rounds of communication whenever up to* $t < n/3$ *of the* n *parties involved are byzantine.*

Proof. We first note that Termination within three rounds follows immediately from the protocol's description. In the following, we analyze each of the properties of Conditional Graded Broadcast separately.

Conditional Validity. We assume P_s is honest and that each honest party P_i joins cGBC with $b_i := 1$. Hence, P_s sends its input x to all parties. Every honest party receives x from P_s, plus up to $t < n - t$ values different from x. Hence, at the end of round 2, each honest party obtains $\mathtt{maj} = x$, and $\#\mathtt{maj} \geq n - t$. Therefore, in round 3 all honest parties send $\mathtt{maj} = x$ to all parties. It follows that every honest party receives at least $n - t$ values x at the end of round 3, and up to $t < n - t$ values different from x. Then, in the Output Determination step, every honest party outputs $(v = x, g = 2)$.

Conditional Graded Consistency. Let P_i and P_j denote two honest parties, and let (v_i, g_i) and (v_j, g_j) denote their outputs.

We first assume $g_i = 2$: P_i has received v_i from at least $n - t$ parties at the end of round 3. Since at least $n - 2t > t$ of these parties are honest, P_j has received v_i from at least $t + 1$ parties at the end of round 3. Assume that P_j has set $\mathtt{maj}' \neq v_i$: then, it has received $\mathtt{maj}'$ at least $t + 1$ times, hence from at least one honest party. This means that, in round 3, an honest party P has sent $v = v_i$, while another honest party P' has sent $v' = \mathtt{maj}'$. Thus, in round 2, P has received v_i from $n - t$ parties, while P' has received v' from $n - t$ parties. However, as $n - t$ parties, hence at least $t + 1$ honest parties have sent v in round 2, we obtain a contradiction: P' has received v' from at most $n - t - 1$ parties. Consequently, P_j has set $\mathtt{maj}' = v_i$. Moreover, since P_i has received v_i from $n - t$ parties in round 3, hence from at least $t + 1$ honest parties, P_j receives v_i at least $t + 1$ times in round 3 and therefore outputs $v_j = v_i$ with grade $g_j \geq 1$.

If $g_j := 2$, we apply a symmetrical argument and obtain that $v_i = v_j$ and $g_i \geq 1$.

It remains to discuss the case where $g_i = g_j = 1$. Assume that $v_i \neq v_j$: this means that P_i has received v_i from at least $t + 1$ parties, hence from at least one honest party P_k at the end of round 3. Similarly, P_j has received v_j from at least $t + 1$ parties, hence from at least one honest party P_l at the end of round 3. Since P_k has sent $\mathtt{maj} = v_i$ in round 3, it has received v_i from $n - t$ parties at the end of round 2. Since at least $n - 2t \geq t + 1$ of these values came from honest parties, P_l has received up to $n - t - 1$ values v_j by the end of round 2, contradicting that it sent $\mathtt{maj} = v_j$ in round 3. Therefore, $v_i = v_j$.

No Honest Participation. Assume that every honest party P_i joins cGBC with input bit $b_i = 0$. Then, the honest parties do not send any messages in round 3, and therefore receive up to t values at the end of round 3. It follows that, in the Output Determination step, every honest party outputs $(\perp, 0)$.

B Missing Proofs

We include the proofs of Theorem 7 and Theorem 8.

Theorem 7. *Let $L \geq \frac{2t}{n-2t}$ be a natural number such that $L^L > t^2$ and $\ell = \lfloor \frac{1}{2} \left(\frac{n-2t}{t}\right)^L L^L \rfloor \geq 2$. Then, if $r = L/2$, there is a $(12r + 11)$-round Byzantine*

Agreement protocol secure against $t < n/3$ corruptions that achieves Agreement except with probability $12 \cdot \left(2 \cdot \left(\frac{n-2t}{t}\right)^2\right)^{-r} \cdot r^{-r}$.

Proof. For the chosen L, Theorem 9, presented in Appendix A, provides a $3 \cdot L$-round $(\ell+1)$-Proxcensus protocol $\texttt{Prox}$ secure against $t < n/3$ corruptions. Moreover, for the chosen L, Corollary 1 provides an (ℓ, t)-ProxCoin protocol $\texttt{ProxCoin}$, also secure against $t < n/3$ corruptions. This protocol ensures termination within $3 \cdot L + 11$ rounds. We may then use these protocols in $\Pi_{\texttt{EE}}^{\ell+1}$, and Theorem 3 provides us with a $(12r+11)$-round Byzantine Agreement protocol that achieves agreement except with probability $\frac{1+2\cdot t}{\ell} = (1+2\cdot t) \cdot \frac{1}{\lfloor \frac{1}{2}\left(\frac{n-2t}{t}\right)^L L^L \rfloor}$.

Since $t \geq 1$, $\ell \geq 2$ and $\lfloor x \rfloor \geq x/2$ whenever $x \geq 2$, the failure probability becomes at most $(6 \cdot t) \cdot 2 \cdot \left(\frac{n-2t}{t}\right)^{-L} L^{-L}$.

Using the assumption that $L^L \geq t^2$ and the notation $r = L/2$, we obtain that the failure probability is at most $12 \cdot \left(2 \cdot \left(\frac{n-2t}{t}\right)^2\right)^{-r} \cdot r^{-r}$, as claimed in the theorem statement.

Theorem 8. *Consider a constant $\varepsilon > 0$, and let $L \geq \frac{1}{\varepsilon} - 1$ be a natural number such that $L^L > t^2$ and $\ell = \lfloor \frac{1}{2}\left(\frac{2\varepsilon}{1-\varepsilon}\right)^L L^L \rfloor \geq 2$. Then, if $r = L/2$, there is a $(12r+17)$-round authenticated Byzantine Agreement protocol secure against $t = (1-\varepsilon)n/2$ corruptions that achieves Agreement except with probability* $12 \cdot \left(2 \cdot \left(\frac{2\varepsilon}{1-\varepsilon}\right)^2\right)^{-r} \cdot r^{-r}$.

Proof. For the chosen L, [32] provides a $3 \cdot L$-round Proxcensus protocol $\texttt{Prox}_{\texttt{auth}}$ with $\ell + 1$ slots secure against $t = (1-\varepsilon)n/2$ corruptions.

Moreover, for the chosen L, Corollary 2 provides an (ℓ, t)-ProxCoin protocol $\texttt{ProxCoin}_{\texttt{auth}}$, also secure against $t = (1-\varepsilon)n/2$ corruptions. This protocol ensures termination within $3 \cdot L + 17$ rounds.

We may then use these protocols in $\Pi_{\texttt{EE}}^{\ell+1}$, and, by Theorem 3, obtain a $(12r+17)$-round Byzantine Agreement protocol that achieves agreement except with probability $\frac{1+2\cdot t}{\ell} = (1+2\cdot t) \cdot \frac{1}{\lfloor \frac{1}{2}\left(\frac{2\varepsilon}{1-\varepsilon}\right)^L L^L \rfloor}$.

Since $t \geq 1$, $\ell \geq 2$ and $\lfloor x \rfloor \geq x/2$ whenever $x \geq 2$, the failure probability becomes at most $(6 \cdot t) \cdot 2 \cdot \left(\frac{2\varepsilon}{1-\varepsilon}\right)^{-L} L^{-L}$. Using the assumption that $L^L \geq t^2$ and the notation $r = L/2$, we obtain that the failure probability is at most $12 \cdot \left(2 \cdot \left(\frac{2\varepsilon}{1-\varepsilon}\right)^2\right)^{-r} \cdot r^{-r}$, as claimed in the theorem statement.